Symbol-Level Deep Learning-Based Decoders for Concatenated Codes Over Insertion/Deletion Channels

E. Uras Kargı and Tolga M. Duman, *Fellow, IEEE*

***Abstract*—Synchronization errors, such as insertions and deletions, pose significant challenges in communication and storage systems, including DNA data storage. Among different alternatives, serially concatenated coding schemes, where a powerful outer code is concatenated with an inner marker code, have proven effective in mitigating such errors. A common method of decoding the inner marker code is to employ the forward-backward algorithm implementing bitwise or symbolwise maximum a posteriori (MAP) detection. Recently, bit-level deep-learning based solutions have also been proposed for decoding marker codes. In this paper, we consider symbol-level deep-learning based decoders, which exploit correlations among adjacent bits to improve the decoding performance, and develop new channel detection algorithms. Unlike existing symbol-level MAP decoding approaches that can combine only two or three consecutive bits, the proposed deep-learning methods extend well beyond this limit, and achieve improved error correction performance. We also develop deep-learning based decoders for insertion/deletion channels further exacerbated by intersymbol interference, motivated by bit-patterned media recording channels and nanopore sequencing. Extensive numerical results show that the proposed symbol-level deep-learning architectures are highly effective for communication over insertion/deletion channels.**

***Index Terms*—Deep learning, deletion channel, insertion channel, channels with synchronization errors, concatenated codes, marker codes, recurrent neural networks, gated recurrent units, GRU, intersymbol interference.**

I. Introduction

CHANNELS with synchronization errors, also known as insertion/deletion/substitution (IDS) channels, are observed in various practical applications. These channels delete bits or symbols from the data stream and/or add random bits to it. Such random deletions or insertions create synchronization problems at the receiver. DNA data storage, one of the emerging technologies for digital storage, is an exemplary application where such errors are observed [2]. As another example, bit-patterned media recording channels also suffer from synchronization errors [3], [4]. Furthermore, synchronization errors are observed in wireless communication systems when the receiver and transmitter clocks are mismatched.

Motivated by the challenges faced by the above technologies, different channel models addressing synchronization errors have been developed. For example, nanopore channels refer to nanoscale pores embedded in membranes that allow the passage of DNA strands [5]. Such channels introduce not only synchronization errors but also intersymbol interference (ISI), as the current stemming from the passage of the strand is affected by the adjacent nucleotides. Another example where ISI is observed along with insertions and deletions is bit-patterned media recording systems [6].

The capacity of channels with independent and identically distributed (i.i.d.) insertions, deletions, and substitutions, has been examined in the literature. It was proved that such channels are information stable; hence, their Shannon capacity exists [7]. However, determination of the capacity has remained elusive, while various upper and lower bounds have been developed [8], [9], [10], [11], [12].

Practical codes for IDS channels have also been proposed, including number-theoretic and algebraic codes, convolutional codes, and concatenated codes. An important class of block codes includes Varshamov-Tenengolts (VT) codes, which are capable of correcting a limited number of deletions and insertions employing suitable syndromes [13], [14]. Convolutional codes, which can be decoded by trellis-based algorithms [15], [16], represent other examples.

An effective solution for correcting random insertion and deletion errors is based on the concatenation of a powerful outer code with an inner marker or watermark code [17], [18]. Marker codes are sequences of bits known to both the receiver and the transmitter and are inserted regularly into the output sequence of the outer code. On the other hand, watermark sequences are added componentwise to the encoded sequence (which is forced to be sparse). The decoder implicitly looks for the positions of marker bits or the background watermark sequences to achieve synchronization over channels with bit gains or bit losses.

Received 4 March 2025; revised 9 October 2025 and 12 December 2025; accepted 31 December 2025. Date of publication 19 January 2026; date of current version 9 February 2026. This work was funded by the European Union through the ERC Advanced Grant 101054904: TRANCIDS. An earlier version of this paper was presented in part at the IEEE International Conference on Communications (ICC), Denver, CO, USA, in June 2024 [DOI: 10.1109/ICC51166.2024.10622561]. The associate editor coordinating the review of this article and approving it for publication was D. Vukobratovic. *(Corresponding author: Tolga M. Duman.)*

E. Uras Kargı is with the Faculty of Electrical Engineering, Mathematics and Computer Science (EEMCS), Delft University of Technology (TU Delft), 2628 XE Delft, The Netherlands (e-mail: ukargi@tudelft.nl).

Tolga M. Duman is with the Department of Electrical and Electronics Engineering, Bilkent University, 06800 Ankara, Türkiye (e-mail: duman@ee.bilkent.edu.tr).

Data is available on-line at https://github.com/Bilkent-CTAR-Lab/Symbol-Level-BIGRU-Decoders-for-IDS-Channels

Digital Identifier 10.1109/TCOMM.2026.3655750

The use of low-density parity check (LDPC) codes as outer codes in the concatenated coding framework has been shown to be highly effective over IDS channels [17], [18], [19]. In particular, it has been shown that marker codes offer improved error correction over channels with synchronization errors compared to watermark codes as inner codes. With this motivation, in this paper, we focus on the concatenation of LDPC codes with marker codes for use over IDS channels.

Recent studies have also explored stand-alone LDPC constructions for synchronization errors. Shibata et al. showed that irregular LDPC codes, with degree profiles tailored to IDS channels, can approach the symmetric information rate without relying on marker sequences [20]. Spatially coupled LDPC (SC-LDPC) codes have likewise been investigated, where windowed or iterative decoding exploits the coupling structure to achieve strong performance on IDS channels, offering a promising alternative to concatenated marker/watermark schemes [21], [22].

Both marker codes and watermark codes over IDS channels are typically decoded using the maximum a posteriori (MAP) decoding algorithm. Specifically, the Bahl-Cocke-Jelinek-Raviv (BCJR) algorithm is used to produce soft information about the bits encoded by the outer code (input to the inner marker or watermark code); this soft information is then employed by the outer code's decoder to obtain estimates of the transmitted bits. While the use of the BCJR algorithm is optimal in terms of generating soft information, its complexity increases drastically when the number of insertions/deletions is large, and it may underperform if the channel parameters (e.g., insertion/deletion probabilities) are not known. It has also been shown that the performance of the MAP detector improves when MAP detection is implemented at the symbol level (considering multiple bits together) rather than the bit level [19]. Symbol-level BCJR decoders perform better than the bit-level ones because they exploit the correlations among the bits input to the channel. While derivations for the bit-level case are easier to obtain, the symbol-level derivations are somewhat complicated, especially when the number of bits comprising each symbol exceeds two or three, and their implementation complexity is significantly higher [19].

Deep learning techniques for channel coding have been a rapidly developing area. The research in this field started with applying basic neural networks, such as Hopfield networks, to decode simple codes with small codeword lengths. Later, multi-layer perceptrons (MLPs) with numerous hidden layers were adopted for decoding, and it was shown that it is possible to achieve MAP detection performance for small codeword lengths [23]. This has evolved into decoding convolutional codes with sequential neural network architectures, especially after the advent of novel architectures such as recurrent neural networks (RNNs) [24], [25]. Nowadays, transformers are being applied to decode channel codes [26], [27]. Neural network-based decoders aim to achieve low-latency and high-performance decoding capabilities. Despite these advancements, decoding with neural networks becomes intractable as codeword lengths increase. Note also that neural networks are employed not only for decoding but also for constructing state-of-the-art codes, such as polar codes, through reinforcement learning [28], [29]. More recent efforts further extend the above ideas to advanced architectures and training strategies, including boosting-based learning for LDPC codes to improve error-floor behavior [30], as well as cross-attention message-passing transformers and foundation models designed for general error-correction tasks [31], [32], [33].

To the best of our knowledge, the only study that combines deep-learning based channel decoders with IDS channels and DNA storage is a recent work introducing an end-to-end autoencoder based IDS-correcting code, which employs disturbance-based discretization and a differentiable IDS channel to effectively handle synchronization errors [34]. With the overall goal of developing low complexity and robust solutions for receiver design for IDS channels, sequential bit-level deep learning architectures, namely, bi-directional gated recurrent units (BI-GRUs), are developed to estimate probabilities of the sequences encoded by outer codes within the framework of concatenated codes [1]. These results show that the error rate performance is competitive with the approaches using the BCJR algorithm-based receivers for decoding the inner code. We further note that, while BCJR-based solution requires the exact channel state information for estimating the log-likelihood ratios (LLRs), bit-level BI-GRUs are more robust and can work effectively even when the channel parameters are not known precisely.

This paper develops symbol-level deep learning decoders to overcome the limits of existing bit- and symbol-level methods, which can only handle very small symbol groupings. We further extend these decoders to concatenated codes for IDS channels with ISI, where conventional trellis-based methods become too complicated for implementation. Such settings arise in certain applications, including bit-patterned media recording and nanopore sequencing.

Key contributions of the paper are summarized as follows:

- BI-GRU-based decoders are developed for channels with synchronization errors, as reported in the preliminary version of this work [1].
- The deep-learning based decoders are extended to symbol level (each symbol comprising multiple bits) so as to take into account the correlations among likelihoods and to improve the performance of bit-level BI-GRU decoders.
- It is shown that while the standard approach to symbol-level detectors is limited to the use of two or three-bit symbols, it is possible to break this barrier and take into account up to six bits as one symbol, and hence to exceed the performance of previously known solutions with reduced complexity.
- Finally, the decoding architecture is extended to handle IDS channels exacerbated by ISI, and it is shown that deep-learning can offer viable solutions even in this case.

The paper is organized as follows. Section II discusses the system model. Section III reviews the bit-level BI-GRU decoders over IDS channels. Section IV proposes symbol-level BI-GRU decoders for concatenated codes over IDS channels. Section V proposes symbol-level BI-GRU decoders adopted for IDS channels further degraded by ISI. Section VI presents some numerical examples demonstrating the performance of

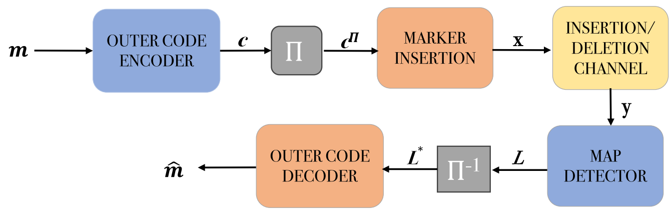


Fig. 1. Block diagram of the system where outer code is concatenated with a marker code.

the proposed architectures. Finally, the paper is concluded in Section VII.

II. System Model

We consider transmission over binary channels susceptible to deletions, insertions, and bit substitutions, referred to as the IDS channel model. Let P_d, P_i, and P_s represent the probabilities of deletion, insertion, and substitution events, respectively. Specifically, each bit in the sequence is either deleted with probability P_d, or replaced by two random bits with probability P_i, or transmitted incorrectly (substituted) with probability $(1-P_d-P_i)P_s$, or transmitted correctly with probability $(1-P_d-P_i)(1-P_s)$. Deletion, insertion, and substitution events are assumed to occur independently across different channel uses. Note that, while different models for insertions exist (see, e.g., [17]), the above channel model (also known as the Gallager model) is used in [35] and many other papers in the literature. The channel model can also be considered as a cascade of an insertion and deletion channel followed by a binary symmetric channel (BSC).

We adopt a concatenated coding scheme which consists of an outer LDPC code, and an inner marker code. The marker code is employed to regain synchronization at the receiver while the powerful outer LDPC code is used to improve the error rate performance. Message vector $\mathbf{m}$ is encoded via the outer code with rate $r_o = k/n$, and then the encoded sequence $\mathbf{c}$ is interleaved. Interleaving can be either in the bit-level or symbol-level depending on which type of estimator is used. A pre-selected marker sequence of length N_m bits is inserted into the coded bit sequence after every N_c bits. The resulting sequence $\mathbf{x} = (x_1, \ldots, x_T)$ with length T is transmitted over IDS channel, and the sequence $\mathbf{y} = (y_1, \ldots, y_R)$ with length R is received. The rate of inner marker code is $r_m = N_c/(N_c + N_m)$, hence the rate of the overall code becomes $r = r_o r_m = \frac{N_c k}{(N_c+N_m)n}$.

Marker sequence and insertion intervals are chosen such that the overall outer rate r_o remains within the achievable-rates corresponding to a given N_c, as calculated in [19] through computation of mutual information by Monte-Carlo techniques. Note that marker sequences containing bit transitions (e.g., "01") are preferred over sequences without transitions (e.g., "00"), since transitions enable insertion or deletion errors to be identified more reliably. Fig. 1 shows the block diagram of a concatenated coding approach where the inner code is a marker code, and the receiver consists of a MAP detector for decoding markers, and an outer code decoder.

III. Bit-Level Deep-Learning Based Decoders for Concatenated Codes Over IDS Channels

In this section, bit-level deep learning decoders (developed in the conference version of this work [1]) for concatenated coding with marker codes as inner codes are reviewed since symbol-level deep learning decoder counterparts are proposed as their extensions. The goal is to provide alternative methods for decoding inner marker codes over IDS channels and to address the limitations of conventional approaches, particularly, their dependence on the channel parameters for optimal performance.

We consider two setups:

1) Two-step decoding: A BI-GRU architecture is first trained to estimate the LLRs of the bits at the input of the marker code. These estimated LLRs are then passed to the outer LDPC decoder, which produces the final message bit estimates.
2) One-shot decoding: Two BI-GRU architectures are cascaded. The first BI-GRU estimates the marker bits, while the second BI-GRU directly decodes the outer code, thereby combining marker decoding and message decoding into a single end-to-end process.

For both setups, the bit-level BI-GRU-based architecture estimates the LLR of each transmitted bit, denoted by $\mathbf{L} = (L_1, \ldots, L_T)$. The estimated LLRs are first de-interleaved at the bit level, and then those corresponding to marker bits are removed because they are irrelevant for the outer code decoder, resulting in the vector $\mathbf{L}^* = (L_1, \ldots, L_n)$. $\mathbf{L}^*$ is passed to the outer code decoder (the sum-product algorithm for the first setup, or deep-learning based decoder for the second setup). The outer code estimates the information bits as $\hat{\mathbf{m}} = (\hat{m}_1, \ldots, \hat{m}_k)$.

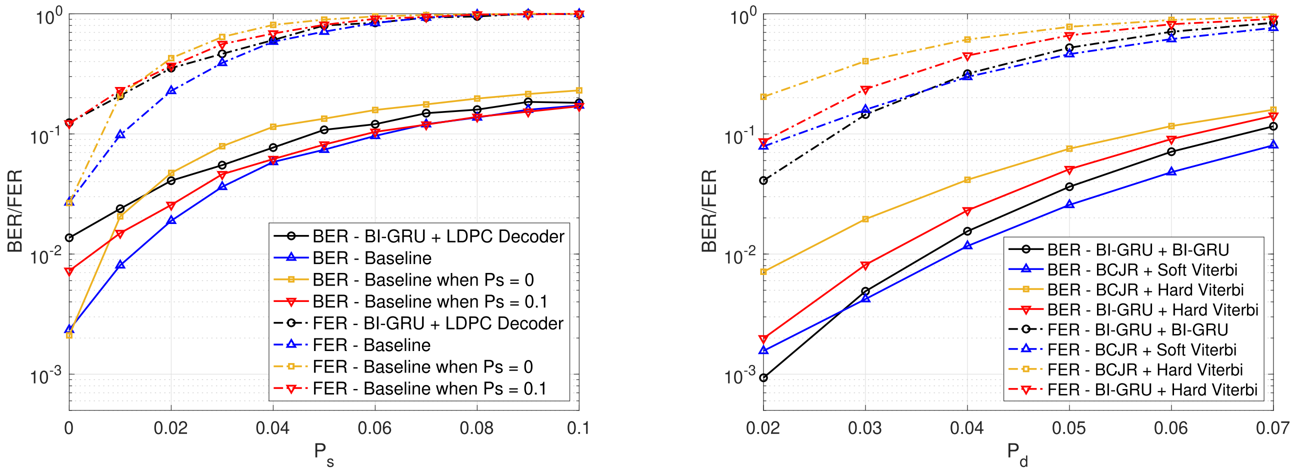
A. Bit-Level BI-GRU Estimators

We now describe the bit-level BI-GRU estimators, which are utilized to compute the LLRs of the transmitted bits. In a forward RNN, the hidden state at a giventime t, $\mathbf{h}_t^{(\mathrm{f})}$, represents the prior information. Let $\mathbf{W}_{\mathrm{rec}}^{(\mathrm{f})}$ and $\mathbf{W}_{\mathrm{in}}^{(\mathrm{f})}$ denote the recurrent weight matrix and the input weight matrix of the forward RNN, respectively. The input vector at time t is defined as $\mathbf{x}_t = (y_{t-\gamma+1}, \ldots, y_t, \ldots, y_{t+\gamma-1}) \in \mathbb{R}^{2\gamma-1}$, which consists of a window of received symbols centered at y_t. This type of input transformation is motivated by BCJR-based estimators, where windowing around the current time index is known to capture almost all relevant information about the received sequence for time step t. Let f denote a generic activation function. We can compute $\mathbf{h}_t^{(\mathrm{f})}$ as follows:

$$\mathbf{h}_t^{(\mathrm{f})} = f(\mathbf{W}_{\mathrm{rec}}^{(\mathrm{f})}\mathbf{h}_{t-1}^{(\mathrm{f})} + \mathbf{W}_{\mathrm{in}}^{(\mathrm{f})}\mathbf{x}_t). \quad (1)$$

In a backward RNN, the hidden state at a given time t represents the information about the sequence to the right of the current input, $\overleftarrow{\mathbf{h}}_t$. Let $\mathbf{W}_{rec}^{(b)}$ and $\mathbf{W}_{in}^{(b)}$ denote the recurrent weight matrix and the input weight matrix of backward RNN, respectively. We can compute $\mathbf{h}_t^{(\mathrm{b})}$ as follows:

$$\mathbf{h}_t^{(\mathrm{b})} = f(\mathbf{W}_{\mathrm{rec}}^{(\mathrm{b})}\mathbf{h}_{t+1}^{(\mathrm{b})} + \mathbf{W}_{\mathrm{in}}^{(\mathrm{b})}\mathbf{x}_t). \quad (2)$$



(a) Setup 1 (two-step decoding)

(b) Setup 2 (one-shot decoding)

Fig. 2. Comparison of bit-level decoders: (a) **Setup 1 (two-step)**—a BI-GRU first estimates LLRs for the marker code, which are then decoded by the outer LDPC decoder; BER/FER vs substitution probability; IDS with $P_d = 0.03$, $P_i = 0$; LDPC $(204, 102)$; marker sequence $[0, 1]$ and $N_c = 10$, (b) **Setup 2 (one-shot)**—two cascaded BI-GRUs estimate marker bits and directly decode the outer code, replacing the conventional outer decoder; Setup 2 (one-shot decoding). BER/FER vs deletion probability for a deletion-only channel ($P_s = 0$), using a rate-$1/2$ convolutional code of length 204 with generators $(5, 7)_{octal}$; marker sequence $[0, 1]$ and $N_c = 10$.

A layer of a BI-RNN [36] combines a forward and a backward RNN described above. A linear layer with a sigmoid activation function is added on top of the final BI-RNN layer to calculate the conditional probability of $x_t = 1$ given $\mathbf{y}$. Let $\mathbf{W}_o$ denote the output weight matrix of the network, $\sigma(x)$ be the sigmoid function, i.e., $\sigma(x) = \frac{1}{1+e^{-x}}$ and $[\,;\,]$ be the concatenation operator. We calculate the output of the network as $P(x_t = 1|\mathbf{y}) = \sigma(\mathbf{W}_o[\mathbf{h}_t^{(\mathrm{f})}; \mathbf{h}_t^{(\mathrm{b})}])$. Then, the LLR of x_t, $L(x_t|\mathbf{y})$ for $t = \{1, \dots, T\}$ is calculated using:

$$L(x_t|\mathbf{y}) = \log \frac{P(x_t = 0|\mathbf{y})}{P(x_t = 1|\mathbf{y})} = \log \frac{1 - \sigma(\mathbf{W}_o[\mathbf{h}_t^{(\mathrm{f})}; \mathbf{h}_t^{(\mathrm{b})}])}{\sigma(\mathbf{W}_o[\mathbf{h}_t^{(\mathrm{f})}; \mathbf{h}_t^{(\mathrm{b})}])}. \tag{3}$$

We implemented the BI-GRU architecture instead of basic RNNs as an LLR estimator for the marker code, since it addresses the issues of vanishing and exploding gradients commonly faced in RNNs, while also significantly improving memory capabilities compared to simple RNNs [37], [38].

B. Training Details for the First Setup

In this subsection, we present the training methodology adopted for the first setup, two-step decoding. The BI-GRU estimators are trained using a supervised learning approach: the labels are the original transmitted sequences $\mathbf{x}$ over an IDS channel, and the inputs are the received sequences $\mathbf{y}$ transformed by $-2\mathbf{y} + 1$. Mapping binary inputs from the channel, originally in $\{0, 1\}$, to $\{-1, +1\}$ prevents information loss that occurs when zeros contribute nothing in linear transformations. This balanced, zero-centered representation preserves the distinction between bit states and enhances both the stability and expressiveness of the neural network during training. Although many transformations are possible, we adopt the mapping $0 \to 1, 1 \to -1$, which is also commonly used in the training of deep-learning based decoders for channel codes [23], [24], [26], [39].

C. Training Details for the Second Setup

In this subsection, we present the training methodology adopted for the second setup, one-shot decoding. In this setup, two BI-GRU networks are trained: one for estimating the markers and one for decoding convolutional codes given the outputs of the marker estimator. For the BI-GRU-based estimator, the training methodology is the same as the one described in Section III-B. For the BI-GRU decoder, the labels are the message sequences $\mathbf{m}$, and the inputs are the LLR vectors $\mathbf{L}^*$ obtained from the BI-GRU-based estimator. Both the BI-GRU estimators and decoders are trained in batches of size B without any fixed training dataset.

D. Numerical Examples

Fig. 2a compares the performance of the proposed decoder with the BCJR-based baseline under Setup 1 (two-step decoding). The results are obtained over a range of substitution probabilities for an IDS channel with $P_d = 0.03$ and $P_i = 0$, using an LDPC code of length $(204, 102)$, marker sequence $(0, 1)$ and $N_c = 10$. It is observed that the bit-level BI-GRU estimator exhibits improved performance in some cases over the baseline decoder. Specifically, when the baseline decoder uses a substitution probability of 0, for a range of deletion probabilities, the decoder outperforms the BCJR algorithm. Notably, in terms of the frame error rate (FER) performance, the bit-level deep learning decoders outperform the mismatched decoder for a wider range of channel parameters.

BI-GRU estimators can work over a range of probabilities, without any change to their input representation or architecture. Moreover, when channel conditions are not known to the receiver, our proposed decoder can outperform baseline methods which require the channel state condition. In addition, the complexity of BI-GRU decoders can be made comparable to that of conventional methods by selecting smaller hidden layer sizes, while maintaining acceptable error-rate performance.

Fig. 2b presents the error-rate performance under Setup 2 (one-shot decoding), where the outer code decoder is replaced by a BI-GRU. For this setup, we evaluate performance over a range of deletion-only channel parameters ($P_i = 0$, and $P_s = 0$), employing a rate-$1/2$ convolutional code with generators $(5, 7)_{octal}$, along with the marker sequence $(0, 1)$ and $N_c = 10$. In this setting, decoding is performed in a single step and we observe that the fully BI-GRU based receiver performs similarly to the baseline decoder, which employs the Viterbi algorithm with soft decision decoding (SDD) as the outer decoder, while a BCJR algorithm is used as the estimator. Given that the training objective of the BI-GRU estimator is binary classification, there appears to be enhanced compatibility with the Viterbi algorithm using hard decision decoding (HDD); the decoder using the BI-GRU estimator along with the Viterbi algorithm employing HDD performs better than the baseline approach paired with the same outer decoding algorithm for all the deletion probabilities considered.

E. Further Discussion and Comparison With Related Work

We elaborate on a closely related work [40], which explores bit-level deep-learning based decoders for coding over IDS channels similar to ours. In this work, carried out in parallel with the preliminary version of this paper [1], the authors propose two distinct setups: a model-based configuration within the framework of forward-backward equations and a model-free setup, where the implementation focuses on end-to-end estimation of LLRs within a framework akin to ours. The differences between our work and [40] can be highlighted as the incorporation of a one-shot decoding approach applied to the receiver in our work, distinct preprocessing steps applied to the received bits, and variations in training methodologies (for instance, utilization of a static dataset versus simulation-based generation of mini-batches during training in our approach).

The results in [40] are compatible with our results on bit-level decoders: the BI-GRU based estimators perform similarly to our trained decoders regarding their performance difference with the baseline decoders realized using forward-backward equations. The model-based decoders utilizing a forward-backward algorithm aimed to estimate P_d, P_s, and P_i, perform better than the BCJR-algorithm based decoders when the channel parameters are not known to the receiver. The authors also experiment with the burst IDS channels of which the deletions and insertions are not independent for different uses and show that without knowing the channel model, one may develop BI-GRU based decoders and argue that their performance in terms of error rates is superior to those of the BCJR-based decoders.

IV. Symbol-Level Deep-Learning Based Decoders

This section proposes a novel deep-learning based method for decoding concatenated codes over IDS channels, focusing on symbol-level decoding. In [1], deep learning decoders were motivated by bit-level decoding methods grounded in the forward-backward algorithm, which considers individual bits. In this section, deep-learning based decoders are adapted to the symbol-level estimators introduced in [19]. Symbol-level decoders take into account the correlations among the bits at the receiver output, which are ignored by the bit-level decoders. This is motivated by the observation that when a bit is affected by an insertion or deletion event, its adjacent bits are more influenced by these errors compared to further away bits. Therefore, when properly designed, symbol-level decoding provides enhancements over bit-level decoding. In this method, decoding is carried out at the symbol level, where the MAP algorithm is used to estimate the posterior probabilities of the symbols that consist of multiple consecutive bits instead of individual bits. While it is theoretically possible to create symbols of any length, prior work primarily focuses on 2- and 3-bit symbol-level estimators, as the MAP detector becomes too complicated for implementation beyond these values.

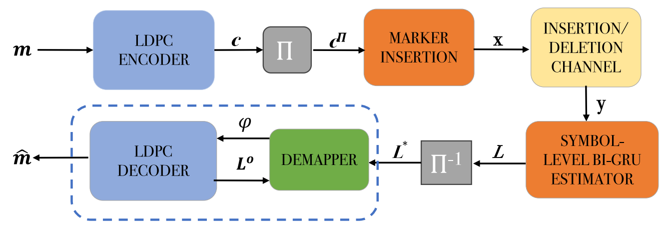


Fig. 3. Block diagram of the proposed decoding framework. An LDPC code is concatenated with a marker code and transmitted over the IDS channel. At the receiver, a symbol-level BI-GRU estimator performs synchronization and demapper and outer code decoder estimates message bits.

Our objectives are threefold: to replace the symbol-level MAP detector with a symbol-level deep-learning based detector to estimate the symbol probabilities, to improve the error rate performance of bit-level deep-learning based detectors, and to extend the limitations of previous symbol-level decoders to handle symbols longer than three bits.

We serially concatenate an LDPC code with a marker code, and develop a BI-GRU architecture to estimate the posterior probabilities of the symbols input to the inner code. The BI-GRU network is employed to regain synchronization at the receiver. This choice is motivated by prior work showing that BI-GRUs can replace the BCJR algorithm for convolutional codes [24], and by related research on IDS channels employing bit-level synchronizers that also adopt BI-GRUs [1], [40]. Moreover, we select the GRU architecture over LSTM networks because they use fewer parameters and a simpler gating mechanism, enabling faster training and reduced risk of overfitting while achieving comparable accuracy on many sequence-modeling tasks [37], [38]. Fig. 3 illustrates the block diagram of the proposed setup.

A. Preliminaries and Encoding/Decoding Procedure

Message vector $\mathbf{m} = (m_1, \ldots, m_k)$ of length k, where $m_t \in \{0, 1\}$ is encoded through the outer (bit-level) code, resulting in $\mathbf{c} = (c_1, \ldots, c_n)$ of length n where $c_t \in \{0, 1\}$. The rate of the outer code is k/n. Then, the encoded sequence $\mathbf{c}$ is grouped into symbols of length

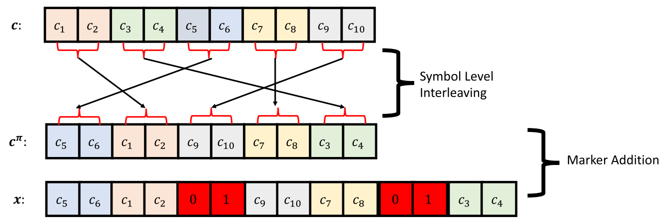


Fig. 4. An illustration of symbol grouping and interleaving, where the marker sequence "01" is inserted after every 4 coded bits ($N_c = 4$).

S, expressed as $\mathbf{c} = (\mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_{n/S})$, where each $\mathbf{s}_i = (c_{(i-1)S+1}, c_{(i-1)S+2}, \ldots, c_{iS})$ for $i \in \{1, 2, \ldots, n/S\}$. Each symbol is a group of S consecutive bits and takes values in $\{0,1\}^S$. Since message vector $\mathbf{m}$ is i.i.d., $P(\mathbf{s}_i) = 1/2^S$ for all $\mathbf{s}_i \in \{0, \ldots, 2^S - 1\}$. We assume that the codeword length n is divisible by S without loss of generality; if not, zero-padding can be applied.

The encoded sequence $\mathbf{c} = (\mathbf{s}_1, \ldots, \mathbf{s}_{n/S})$ is interleaved at the symbol level, resulting in $\mathbf{c}^\pi$. Symbol-level interleaving permutes groups of bits rather than individual bits, and the interleaver is known to both the receiver and the transmitter. The interleaver is chosen as a random permutation. Then, a pre-selected marker with length N_m is inserted after every N_c coded bits, resulting in the overall code $\mathbf{x} = (x_1, \ldots, x_T)$ where $T = n + N_m(n/N_c)$ denotes the total number of transmitted bits. This process is illustrated in Fig. 4 which highlights the symbol-level interleaving operation.

The overall codeword is transmitted over the IDS channel and $\mathbf{y} = (y_1, \ldots, y_R)$ is received, where the length of the received sequence, R, is a random variable. The proposed BI-GRU based estimator computes $P(\mathbf{s}_t \mid \mathbf{y})$ for each $\mathbf{s}_t \in \{0, \ldots, 2^S - 1\}$, yielding a total of n/S terms. Then, the S-dimensional vector at time step t, $L_t = \big(P(\mathbf{s}_t = 0 \mid \mathbf{y}), \ldots, P(\mathbf{s}_t = 2^S - 1 \mid \mathbf{y})\big)$, is formed. For example, if $S = 2$,

$$\mathbf{L}_t = \begin{pmatrix} P(\mathbf{s}_t = (0,0) \mid \mathbf{y}) \\ P(\mathbf{s}_t = (0,1) \mid \mathbf{y}) \\ P(\mathbf{s}_t = (1,0) \mid \mathbf{y}) \\ P(\mathbf{s}_t = (1,1) \mid \mathbf{y}) \end{pmatrix}.$$

After this calculation the vector consisting of posterior probabilities is formed, $\mathbf{L} = (L_1, \ldots, L_{n/S})$. These values are de-interleaved at the symbol level, resulting in the vector $\mathbf{L}^\pi = (L_1, \ldots, L_{n/S})$. $\mathbf{L}^\pi$ is then provided as input to a combination of the outer code and the demapper module. There is a local exchange of information between the demapper and the outer-code decoder: after every N_o iterations of the outer-code decoder, the LLRs calculated by the outer-code decoder, denoted by $\mathbf{L}^o = (L_1, \ldots, L_n)$, where $L_t = \log \frac{P(x_t=0|\mathbf{y})}{P(x_t=1|\mathbf{y})}$, are passed to the demapper. Then, the demapper produces $\Phi = (\varphi_1, \ldots, \varphi_n)$. The demapper exploits the soft information from the outer-code decoder to update its estimates, and its output Φ is then fed back to the outer code decoder. After a total of M such local exchanges (i.e., MN_o iterations of the outer code decoder), hard decisions on the message bits are obtained, $\hat{\mathbf{m}} = (\hat{m}_1, \ldots, \hat{m}_k)$, where $\hat{m}_t \in \{0,1\}$. We note that increasing M generally improves performance because the outer decoder benefits from more refined soft information; however, larger M also increases computational complexity due to the more frequent usage of the demapper module.

For $S = 2$, the demapper is defined as follows [19], [41], where $x_{2t-1} \in \{0,1\}$ and for $t \in \{1, \ldots, n/2\}$,

$$\begin{aligned} \varphi(x_{2t-1}) &\triangleq P(\mathbf{y} \mid x_{2t-1}) \\ &= \sum_{\mathbf{s}_t} P(\mathbf{y}, \mathbf{s}_t \mid x_{2t-1}) \\ &= \sum_{\mathbf{s}_t} P(\mathbf{s}_t \mid x_{2t-1}) P(\mathbf{y} \mid \mathbf{s}_t) \\ &= \sum_{\mathbf{s}_t} P(\mathbf{y} \mid \mathbf{s}_t) \sum_{x_{2t} \in \{0,1\}} P(\mathbf{s}_t, x_{2t} \mid x_{2t-1}). \end{aligned}$$

We expand the term $P(\mathbf{s}_t, x_{2t} \mid x_{2t-1})$ to continue the expression:

$$\begin{aligned} \varphi(x_{2t-1}) &= \sum_{\mathbf{s}_t} P(\mathbf{y} \mid \mathbf{s}_t) \sum_{x_{2t} \in \{0,1\}} P(\mathbf{s}_t \mid x_{2t-1}, x_{2t}) \\ &\quad P(x_{2t} \mid x_{2t-1}) \\ &= \sum_{\mathbf{s}_t} P(\mathbf{y} \mid \mathbf{s}_t) \sum_{x_{2t} \in \{0,1\}} P(\mathbf{s}_t \mid x_{2t-1}, x_{2t}) \\ &\quad P(x_{2t}) \end{aligned}$$

where the last equality follows from the fact that the message bits are i.i.d., i.e., $P(x_{2t} \mid x_{2t-1}) = P(x_{2t})$.

$$\begin{aligned} \varphi(x_{2t-1}) &= \sum_{\mathbf{s}_t} P(\mathbf{y} \mid \mathbf{s}_t) \Big(P(\mathbf{s}_t \mid x_{2t-1}, x_{2t} = 0) P(x_{2t} = 0) \\ &\qquad + P(\mathbf{s}_t \mid x_{2t-1}, x_{2t} = 1) P(x_{2t} = 1) \Big) \\ &= \sum_{\mathbf{s}_t} P(\mathbf{y} \mid \mathbf{s}_t) \Big(\mathbb{I}\{\mathbf{s}_t = (x_{2t-1}, x_{2t} = 0)\} \frac{e^{L_{2t}}}{1 + e^{L_{2t}}} \\ &\qquad + \mathbb{I}\{\mathbf{s}_t = (x_{2t-1}, x_{2t} = 1)\} \frac{1}{1 + e^{L_{2t}}} \Big) \end{aligned}$$

where $P(\mathbf{y} \mid \mathbf{s}_t)$ is obtained from the symbol-level BI-GRU (or MAP) estimator, $P(x_{2t})$ is derived from the extrinsic LLRs of the outer LDPC decoder, $L_t = \log \frac{P(x_t=0|\mathbf{y})}{P(x_t=1|\mathbf{y})}$, i.e.,

$$\begin{aligned} P(x_{2t} = 0) &\approx P(x_{2t} = 0 \mid \mathbf{y}) = \frac{e^{L_{2t}}}{1 + e^{L_{2t}}}, \\ P(x_{2t} = 1) &\approx P(x_{2t} = 1 \mid \mathbf{y}) = \frac{1}{1 + e^{L_{2t}}}, \end{aligned}$$

and $P(\mathbf{s}_t \mid x_{2t-1}, x_{2t}) = \mathbb{I}\{\mathbf{s}_t = (x_{2t-1}, x_{2t})\}$ is an indicator function, i.e.,

$$\mathbb{I}\{\mathbf{s}_t = (x_{2t-1}, x_{2t})\} = \begin{cases} 1, & \text{if } \mathbf{s}_t = (x_{2t-1}, x_{2t}), \\ 0, & \text{otherwise.} \end{cases}$$

Similarly, $\varphi(x_{2t})$ is calculated in the same way as $\varphi(x_{2t-1})$, where $x_{2t} \in \{0,1\}$ and for $t \in \{1, \ldots, n/2\}$,

$$\begin{aligned} \varphi(x_{2t}) &\triangleq P(\mathbf{y} \mid x_{2t}) \\ &= \sum_{\mathbf{s}_t} P(\mathbf{y} \mid \mathbf{s}_t) \sum_{x_{2t-1} \in \{0,1\}} P(\mathbf{s}_t \mid x_{2t-1}, x_{2t}) \\ &\quad P(x_{2t-1}) \end{aligned}$$

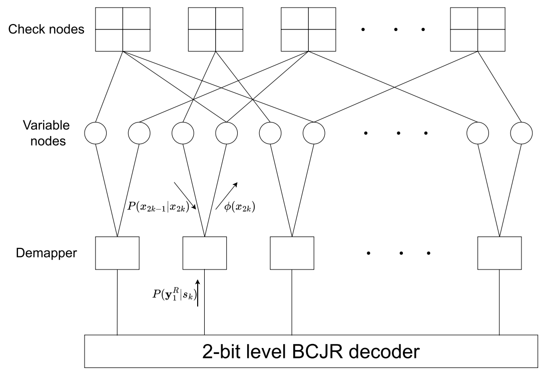


Fig. 5. Illustration of the demapper module for $S = 2$. The demapper receives symbol-level posterior probabilities $\mathbf{L} = (L_1, \ldots, L_{n/S})$ from the BI-GRU (or MAP) estimator and extrinsic LLRs $\mathbf{L}^o$ from the LDPC decoder, where $L_t^o = \log \frac{P(x_t=0|\mathbf{y})}{P(x_t=1|\mathbf{y})}$. It computes updated probabilities for each coded bit, $\Phi = (\varphi_1, \ldots, \varphi_n)$, which are iteratively exchanged with the outer decoder after every N_o iterations.

$$
\begin{aligned}
&= \sum_{\mathbf{s}_t} P(\mathbf{y} \mid \mathbf{s}_t)\Big(P(\mathbf{s}_t \mid x_{2t-1} = 0, x_{2t})P(x_{2t-1} = 0) \\
&\quad + P(\mathbf{s}_t \mid x_{2t-1} = 1, x_{2t})P(x_{2t-1} = 1)\Big) \\
&= \sum_{\mathbf{s}_t} P(\mathbf{y} \mid \mathbf{s}_t)\Big(\mathbb{I}\{\mathbf{s}_t = (x_{2t-1} = 0, x_{2t})\}\frac{e^{L_{2t-1}}}{1 + e^{L_{2t-1}}} \\
&\quad + \mathbb{I}\{\mathbf{s}_t = (x_{2t-1} = 1, x_{2t})\}\frac{1}{1 + e^{L_{2t-1}}}\Big).
\end{aligned}
$$

Fig. 5 shows an illustration of the demapper module for $S = 2$. Demapper for $S = 3$ performs similar steps, for $t \in \{1, \ldots, n/3\}$, namely,

$$
\begin{aligned}
\varphi(x_{3t}) &\triangleq P(\mathbf{y} \mid x_{3t}) \\
&= \sum_{\mathbf{s}_t} P(\mathbf{y}, \mathbf{s}_t \mid x_{3t}) \\
&= \sum_{\mathbf{s}_t} P(\mathbf{s}_t \mid x_{3t})P(\mathbf{y} \mid \mathbf{s}_t, x_{3t}) \\
&= \sum_{\mathbf{s}_t} P(\mathbf{s}_t \mid x_{3t})P(\mathbf{y} \mid \mathbf{s}_t) \\
&= \sum_{\mathbf{s}_t} P(\mathbf{y} \mid \mathbf{s}_t) \sum_{x_{3t-2}} \sum_{x_{3t-1}} P(\mathbf{s}_t, x_{3t-2}, x_{3t-1} \mid x_{3t}) \\
&= \sum_{\mathbf{s}_t} P(\mathbf{y} \mid \mathbf{s}_t) \sum_{x_{3t-2}} \sum_{x_{3t-1}} P(x_{3t-2} \mid x_{3t-1}, x_{3t}) \\
&\quad P(x_{3t-1} \mid x_{3t})P(\mathbf{s}_t \mid x_{3t-2}, x_{3t-1}, x_{3t}) \\
&= \sum_{\mathbf{s}_t} P(\mathbf{y} \mid \mathbf{s}_t) \sum_{x_{3t-2}} P(x_{3t-2}) \sum_{x_{3t-1}} P(x_{3t-1}) \\
&\quad P(\mathbf{s}_t \mid x_{3t-2}, x_{3t-1}, x_{3t}).
\end{aligned}
$$

where $P(\mathbf{y} \mid \mathbf{s}_t)$ is obtained from the symbol-level BI-GRU (or MAP) estimator, $P(x_{3t-2})$ and $P(x_{3t-1})$ are derived from the extrinsic LLRs of the outer LDPC decoder as in the case of $S = 2$, and $P(\mathbf{s}_t \mid x_{3t-2}, x_{3t-1}, x_{3t}) = \mathbb{I}\{\mathbf{s}_t = (x_{3t-2}, x_{3t-1}, x_{3t})\}$ is an indicator function.

B. Symbol-Level Bi-GRU Decoders

In this section, we introduce symbol-level BI-GRU based estimators. These detectors estimate $P(\mathbf{s}_t \mid \mathbf{y})$ at each time step $t \in \{1, \ldots, n/S\}$ for every symbol $\mathbf{s}_t \in \{0, \ldots, 2^S - 1\}$. This estimator works slightly differently from the symbol-level MAP detector in the sense that it directly estimates the posterior term without calculating the intermediate likelihoods, $P(\mathbf{y} \mid \mathbf{s}_t)$ as done in [19].

A BI-GRU architecture with N_{GRU} stacked layers is employed. Let $\odot$ denote the element-wise multiplication operator for two vectors of length n, $\mathbf{x} \odot \mathbf{y} = (x_1y_1, \ldots, x_ny_n)$. The equations for the j^{th} forward GRU layer for $j \in \{1, \ldots, N_{\text{GRU}}\}$ are as follows [37]:

$$
\begin{aligned}
\mathbf{z}_t^{(j,\text{f})} &= \sigma\big(\mathbf{W}_z^{(j,\text{f})}\mathbf{h}_t^{(j-1,\text{f})} + \mathbf{U}_z^{(j,\text{f})}\mathbf{h}_{t-1}^{(j,\text{f})} + \mathbf{b}_z^{(j,\text{f})}\big), \\
\mathbf{r}_t^{(j,\text{f})} &= \sigma\big(\mathbf{W}_r^{(j,\text{f})}\mathbf{h}_t^{(j-1,\text{f})} + \mathbf{U}_r^{(j,\text{f})}\mathbf{h}_{t-1}^{(j,\text{f})} + \mathbf{b}_r^{(j,\text{f})}\big), \\
\tilde{\mathbf{h}}_t^{(j,\text{f})} &= \tanh\Big(\mathbf{W}_h^{(j,\text{f})}\mathbf{h}_t^{(j-1,\text{f})} + \mathbf{U}_h^{(j,\text{f})}\big(\mathbf{r}_t^{(j,\text{f})} \odot \mathbf{h}_{t-1}^{(j,\text{f})}\big) \\
&\quad + \mathbf{b}_h^{(j,\text{f})}\Big), \\
\mathbf{h}_t^{(j,\text{f})} &= \mathbf{z}_t^{(j,\text{f})} \odot \mathbf{h}_{t-1}^{(j,\text{f})} + \big(1 - \mathbf{z}_t^{(j,\text{f})}\big) \odot \tilde{\mathbf{h}}_t^{(j,\text{f})}.
\end{aligned}
$$

The forward GRU parameters are given by $\mathbf{W}_{\{\cdot\}}^{(j,\text{f})} \in \mathbb{R}^{d_{\text{GRU}} \times d_{\text{in}}^{(j)}}$ and $\mathbf{U}_{\{\cdot\}}^{(j,\text{f})} \in \mathbb{R}^{d_{\text{GRU}} \times d_{\text{GRU}}}$, with bias vectors $\mathbf{b}_{\{\cdot\}}^{(j,\text{f})} \in \mathbb{R}^{d_{\text{GRU}}}$ where d_{GRU} denotes the hidden size of each GRU layer, and $d_{\text{in}}^{(j)}$ denotes the length of the input vector to the j^{th} layer. Note that we define $\mathbf{h}_t^{(0,\text{f})} \triangleq \mathbf{x}_t$ as the input to the first forward GRU layer; $d_{\text{in}}^{(1)}$ corresponds to the length of $\mathbf{x}_t$, and $d_{\text{in}}^{(j)} = d_{\text{GRU}}$ for $j > 1$.

Similarly, the backward GRU is obtained by reversing the temporal dependencies, replacing $\mathbf{h}_{t-1}^{(j,\text{f})}$ with $\mathbf{h}_{t+1}^{(j,\text{b})}$, and using the corresponding parameter sets $\mathbf{W}_{\{\cdot\}}^{(j,\text{b})}$, $\mathbf{U}_{\{\cdot\}}^{(j,\text{b})}$, and $\mathbf{b}_{\{\cdot\}}^{(j,\text{b})}$.

The hidden state vectors from the forward and backward GRUs at time step t, $\mathbf{h}_t^{(j,\text{f})}$ and $\mathbf{h}_t^{(j,\text{b})}$, are concatenated and fed into a multilayer perceptron (MLP) consisting of N_{MLP} fully connected layers. The hidden layer dimensions of the MLP are denoted by $\mathbf{d}_{\text{MLP}} = (d_{\text{MLP}}^{(1)}, \ldots, d_{\text{MLP}}^{(N_{\text{MLP}})})$, and each layer $j \in \{1, \ldots, N_{\text{MLP}}\}$ is parameterized by a weight matrix $\mathbf{W}_{\text{MLP}}^{(j)} \in \mathbb{R}^{d_{\text{MLP}}^{(j)} \times d_{\text{MLP}}^{(j-1)}}$ and a bias vector $\mathbf{b}_{\text{MLP}}^{(j)} \in \mathbb{R}^{d_{\text{MLP}}^{(j)}}$, where $d_{\text{MLP}}^{(0)} = 2d_{\text{GRU}}$ corresponds to the BI-GRU output dimension.

Let $\mathbf{a}_t^{(0)} = [\mathbf{h}_t^{(N_{\text{GRU}},\text{f})}; \mathbf{h}_t^{(N_{\text{GRU}},\text{b})}]$ denote the input to the MLP. For each MLP layer, the forward propagation is given by

$$
\mathbf{a}_t^{(j)} = \text{ReLU}\Big(\mathbf{W}_{\text{MLP}}^{(j)}\mathbf{a}_t^{(j-1)} + \mathbf{b}_{\text{MLP}}^{(j)}\Big), \quad j = 1, \ldots, N_{\text{MLP}}.
$$

Following the hidden layers, an output layer parameterized by $\mathbf{W}_o \in \mathbb{R}^{2^S \times d_{\text{MLP}}^{(N_{\text{MLP}})}}$ and $\mathbf{b}_o \in \mathbb{R}^{2^S}$ produces the logit vector $\mathbf{z}_t \in \mathbb{R}^{2^S}$ corresponding to the 2^S possible symbols,

$$
\mathbf{z}_t = \mathbf{W}_o\mathbf{a}_t^{(N_{\text{MLP}})} + \mathbf{b}_o, \qquad \mathbf{z}_t = (z_{t,0}, \ldots, z_{t,2^S-1}).
$$

Applying the softmax function yields the posterior probability of symbol $\mathbf{s}_t = i$ given the received signal $\mathbf{y}$,

$$
P(\mathbf{s}_t = i \mid \mathbf{y}) = \frac{\exp\big(z_{t,i}/\tau\big)}{\sum_{j=0}^{2^S-1} \exp\big(z_{t,j}/\tau\big)},
$$

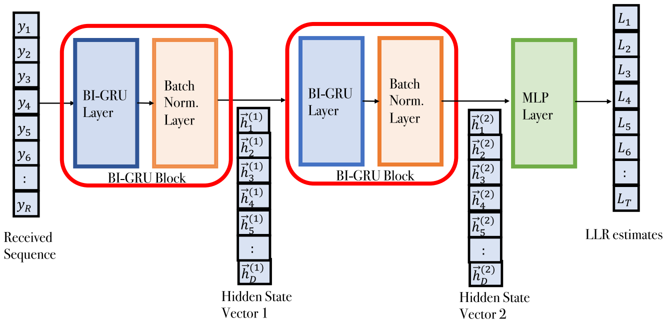


Fig. 6. A BI-GRU detector for bit LLRs.

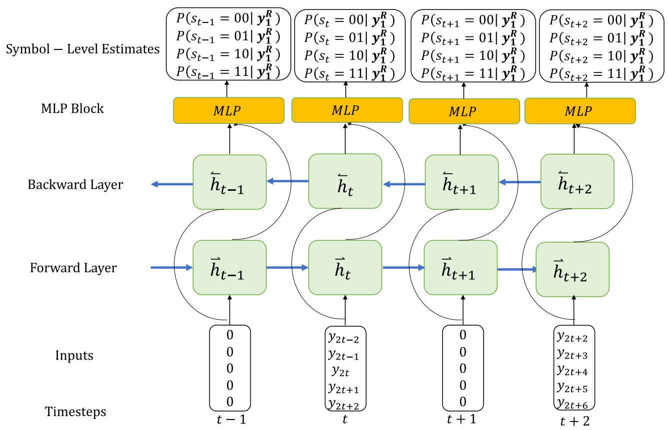


Fig. 7. The block diagram for the symbol-level BI-GRU estimator when $\gamma = 2$, $S = 2$, and $N_c = 2$.

where τ is the temperature parameter that controls the sharpness of the output distribution. Fig. 6 illustrates the combined BI-GRU and MLP architecture.

Another hyperparameter γ is used to convert the received vector $\mathbf{y} = (y_1, \ldots, y_R)$ into inputs to the BI-GRU detector at step t: The input to the BI-GRU detector is $\mathbf{X}_t = (y_{t-\gamma}, \ldots, y_t, \ldots, y_{t+\gamma})$ if t corresponds to the marker index; otherwise, the input at time step t is the all-zero vector of length $2\gamma + 1$, $\mathbf{0}_{2\gamma+1}$. A time step t is a marker index if $t = j\left(\frac{N_c}{S} + 1\right)$ for $j \in \left\{1, 2, \ldots, \frac{n}{N_c}\right\}$. This input representation is employed by analyzing the forward and backward equations of the symbol-level MAP decoder and noting that marker sequences are compared with the received channel bits at the positions where the markers are inserted. The expression for constructing inputs, for $t \in \{1, \ldots, n/S\}$ is given as,

$$\mathbf{X}_t = \begin{cases} (y_{t-\gamma}, \ldots, y_t, \ldots, y_{t+\gamma}), & \text{if } t = j\left(\frac{N_c}{S} + 1\right), \\ \mathbf{0}_{2\gamma+1}, & \text{otherwise,} \end{cases} \tag{4}$$

where $j \in \left\{1, 2, \ldots, \frac{n}{N_c}\right\}$. Fig. 7 shows the block diagram for the symbol-level BI-GRU estimator when $\gamma = 2$, $S = 2$, and $N_c = 2$.

C. Training Details

The BI-GRU estimator is trained in batches of size B. For each batch, a set of random message vectors $\mathbf{M} = \{\mathbf{m}^{(1)}, \ldots, \mathbf{m}^{(B)}\}$ is generated, where each $\mathbf{m}^{(i)}$ is encoded with the outer and inner codes to produce the corresponding codeword $\mathbf{x}^{(i)}$. These codewords are transmitted through the IDS channel, yielding the received sequences $\mathbf{Y} = \{\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(B)}\}$, where $\mathbf{y}^{(i)} = (y_1^{(i)}, \ldots, y_{R_i}^{(i)})$ with random length R_i. During training, the pairs $(\mathbf{y}^{(i)}, \mathbf{m}^{(i)})$ serve as input–label examples for the BI-GRU estimator. Categorical cross-entropy (CE) loss is used as the loss function, where $\hat{y}_{t,i} = P(\mathbf{s}_t = i \mid \mathbf{y})$ denotes the model prediction and $y_{t,i}$ is a binary indicator that equals 1 if the true symbol at time step t corresponds to class i, and 0 otherwise, i.e.,

$$\mathcal{L}_{\text{CE}} = -\frac{1}{(n/S)B} \sum_{b=1}^{B} \sum_{t=1}^{n/S} \sum_{i=0}^{2^S-1} y_{t,i}^{(b)} \log(\hat{y}_{t,i}^{(b)}). \tag{5}$$

The channel conditions of the IDS channel for creating training batches $(\mathbf{y}^{(i)}, \mathbf{m}^{(i)})$ are described by the discrete sets $\mathbf{P}_d^{\text{train}} = \{P_d^1, \ldots, P_d^{M_d}\}$, $\mathbf{P}_s^{\text{train}} = \{P_s^1, \ldots, P_s^{M_s}\}$, and $\mathbf{P}_i^{\text{train}} = \{P_i^1, \ldots, P_i^{M_i}\}$. For each training pair, the channel parameters (P_d, P_s, P_i) are drawn independently from these sets. These channel conditions are treated as hyperparameters during training, and they must be chosen carefully to ensure that the BI-GRU estimator generalizes across the desired range of operating scenarios. For instance, if the target operating regime corresponds to deletion probabilities in the range $P_d \in [0.02, 0.05]$, then the training batches should be generated using values of P_d sampled within this interval so that the network learns to generalize across all relevant conditions. However, if the training also includes unrealistically large deletion probabilities, such as $P_d = 0.2$ for the above example, which are never encountered in practice, the network may fail to converge. In other words, this approach is effective when the training channel conditions are chosen appropriately.

We use the Adam optimizer [42] with a staircase learning rate decay [43], where the learning rate is exponentially decayed by parameter D every N_{wd} steps during training. We use the default hyperparameters of the Adam optimizer. Batch normalization layers are added between the BI-GRU layers to stabilize and accelerate training [44].

D. Complexity Analysis

In this section, we provide a computational complexity analysis of the BI-GRU–based detector and compare it with the MAP detector implemented via the forward–backward algorithm.

We begin by analyzing the computational complexity of the BI-GRU estimator. A BI-GRU consists of several BI-GRU layers, followed by an MLP applied on top of the final BI-GRU layer. We first consider the complexity of the BI-GRU layers. A GRU layer consists of different gates: $\mathbf{z}_t^{(j)}$, $\mathbf{r}_t^{(j)}$, $\tilde{\mathbf{h}}_t^{(j)}$, and $\mathbf{h}_t^{(j)}$ for layers $j \in \{1, \ldots, N_{\text{GRU}}\}$ and time steps $t \in \{1, \ldots, n/S\}$. For the gates $\mathbf{z}_t^{(j)}$ and $\mathbf{r}_t^{(j)}$, a total of $2d_{\text{GRU}}^2$ multiplications and $2d_{\text{GRU}}^2$ additions are needed, respectively. For the calculation of $\tilde{\mathbf{h}}_t^{(j)}$, $2d_{\text{GRU}}^2 + d_{\text{GRU}}$ multiplications and $2d_{\text{GRU}}^2$ additions are required, while the computation of $\mathbf{h}_t^{(j)}$ requires $2d_{\text{GRU}}$ multiplications and $2d_{\text{GRU}}$ additions. Thus, each GRU layer requires $6d_{\text{GRU}}^2 + 3d_{\text{GRU}} \approx 6d_{\text{GRU}}^2$ multiplications and $6d_{\text{GRU}}^2 + 2d_{\text{GRU}} \approx 6d_{\text{GRU}}^2$ additions. Since the backward GRU layer has the same complexity as the forward GRU layer, these values are doubled. With N_{GRU} layers and n/S time steps, the BI-GRU therefore requires approximately $N_{\text{GRU}}\frac{n}{S}(12d_{\text{GRU}}^2+6d_{\text{GRU}})$ multiplications and $N_{\text{GRU}}\frac{n}{S}(12d_{\text{GRU}}^2+4d_{\text{GRU}})$ additions.

On top of the forward and backward layers, an MLP with N_{MLP} layers is applied. Assuming all hidden layers have size $d_{\text{MLP}}^{(i)}=d_{\text{MLP}}$ for simplicity, each hidden layer requires d_{MLP}^2 multiplications and d_{MLP}^2 additions per time step, resulting in a total cost of $N_{\text{MLP}}\frac{n}{S}d_{\text{MLP}}^2$ for this part. The final output layer has size $2^S\times d_{\text{MLP}}$, which introduces additional $2^S d_{\text{MLP}}$ multiplications and 2^S additions per time step. Therefore, the total computational cost of the MLP becomes $\frac{n}{S}\left(N_{\text{MLP}}d_{\text{MLP}}^2+2^S d_{\text{MLP}}\right)$ multiplications and $\frac{n}{S}\left(N_{\text{MLP}}d_{\text{MLP}}^2+2^S\right)$ additions.

Combining the BI-GRU and MLP contributions, the total computational complexity of the BI-GRU estimator is approximately $\frac{n}{S}\left(12N_{\text{GRU}}d_{\text{GRU}}^2+N_{\text{MLP}}d_{\text{MLP}}^2+2^S d_{\text{MLP}}\right)$ multiplications and $\frac{n}{S}\left(12N_{\text{GRU}}d_{\text{GRU}}^2+N_{\text{MLP}}d_{\text{MLP}}^2+2^S\right)$ additions.

We now proceed with the complexity analysis of the MAP detector. Symbol-level MAP detectors realized using the BCJR algorithm require both forward and backward recursions. At each time instance t, where $t\in\{1,\ldots,T/S\}$, approximately $3^S 2R$ multiplications and additions are needed for both the forward and the backward recursions. Since there are T/S time instances, the total number of operations, considering both recursions, is approximately $\frac{T}{S}4\cdot 3^S R$. Because $R\approx T$ due to $P_d\approx P_i$, which slightly changes the received sequence length, the total number of additions and multiplications becomes approximately $4\cdot 3^S\frac{T^2}{S}$.

The key difference between the two complexity expressions lies in their exponential dependence on S. For the MAP detector, the dominant term $T^2 3^S$ grows exponentially in S, leading to a rapid increase in computational cost as S increases. In contrast, the BI-GRU estimator contains the term 2^S, which grows more slowly and appears only as a multiplicative factor of $d_{\text{MLP}}\frac{n}{S}$ and becomes non-dominant compared to the term $N_{\text{MLP}}d_{\text{MLP}}^2$. As a result, the overall complexity of the BI-GRU decoder scales much more gently with S.

Furthermore, the hyperparameter d_{GRU} in the BI-GRU can be selected much smaller than T when T is large. When $d_{\text{GRU}}<T$, the computational complexities of the symbol-level BI-GRU estimator and the symbol-level MAP detector become comparable. For longer codewords (larger than 500), d_{GRU} can be chosen significantly smaller than T, which reduces the overall complexity of the BI-GRU decoder. For large n, the BI-GRU decoder can achieve a substantially lower complexity than the symbol-level MAP algorithm.

Finally, we note that the complexity of BI-GRU can be further reduced through pruning techniques, which effectively decrease the computational cost, although such techniques were not considered in the above analysis. In a similar manner, the BCJR algorithm may also benefit from reduced-state methods that lower its complexity, and our remark on pruning should therefore be viewed as a parallel observation highlighting modern approaches for neural architectures rather than as a direct comparison.

V. Further Extensions

In this section, we focus on channels with insertions and deletions, further exacerbated by intersymbol interference. In other words, ISI coexists with insertions and deletions. There are several examples of related channel models. For instance, the authors in [6] consider synchronization errors and ISI in bit-patterned media recording systems, leading to IDS channels with ISI. This channel model can be considered as cascade of an IDS channel followed by an ISI channel. Another related scenario is the nanopore sequencing channel [5], which also deals with ISI. Nanopore sequencing involves encoding digital data into DNA strands and passing these sequences through a nanoscale biological membrane. As nucleotides pass through the membrane, they generate current levels that the sequencer uses to identify each nucleotide. However, these current levels are affected by both the current and preceding nucleotides, leading to ISI from earlier bits.

Our objective is to adopt the proposed symbol-level BI-GRU decoders to estimate the information bits encoded by the outer code in the concatenated coding setup over IDS channels with ISI. We demonstrate that deep-learning based estimators provide effective solutions in scenarios where deriving explicit analytical expressions is not practical, and implementation complexities of traditional trellis-based MAP detectors are too high. In other words, we are interested in exploring the potential of deep-learning based decoders in even more complex settings than IDS-only channels. Specifically, we adopt the channel model in [6]; however, the general ideas can also be employed for other scenarios, including nanopore sequencing.

A. An IDS Channel With ISI

We consider an IDS channel with ISI, which is modeled as cascade of an IDS channel with P_i, P_d, and P_s, denoting insertion, deletion, and substitution probabilities, respectively, (described in Section II), and an ISI channel. The codeword sequence, $\mathbf{x}=(x_1,x_2,\ldots,x_T)$ where each bit $x_i\in\{-1,+1\}$, is transmitted first through the IDS channel. Due to random insertions and deletions, this sequence is transformed into a new sequence $\tilde{\mathbf{x}}=(\tilde{x}_1,\tilde{x}_2,\ldots,\tilde{x}_R)$ where each bit $\tilde{x}_i\in\{-1,+1\}$. This sequence experiences ISI characterized by the channel taps $\mathbf{h}=(h_0,\ldots,h_{N_T-1})$, where the number of channel taps is N_T and $h_i\in\mathbb{R}$. The channel taps $\mathbf{h}$ are normalized such that $\sum_{l=0}^{N_T-1}|h_l|^2=1$. Additionally, the transmitted signal is affected by additive white Gaussian noise (AWGN) z_k, which has zero mean and variance of $\sigma^2=\frac{N_0}{2}$. The signal-to-noise ratio (SNR) is defined as $\text{SNR}=\frac{1}{N_0}$. The input-output relationship is given by:

$$y_k=\sum_{l=0}^{N_T-1}h_l\tilde{x}_{k-l}+z_k,$$

for $k\in\{1,2,\ldots,R\}$. The received sequence is denoted by $\mathbf{y}=(y_1,\ldots,y_R)$. The overall channel model is illustrated in Fig. 8.

We assume that the ISI channel response $\mathbf{h}$ is fixed, which is consistent with recording channel models where such responses are commonly observed [45]. We note, however,

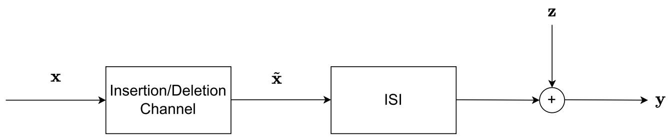


Fig. 8. IDS channel with ISI.

that the proposed BI-GRU based decoder does not require explicit knowledge of $\mathbf{h}$ to regain synchronization. Even under moderate mismatch between the assumed and actual channel response, the decoder can adapt and maintain reliable performance, as it learns directly from data.

B. Trellis Representation for IDS Channels With ISI

In [6], a method is proposed to calculate the information rates of IDS channels with ISI using the forward-backward algorithm. A trellis representation is constructed to jointly represent the states due to insertions (and/or deletions) and ISI. For instance, for the case with deletions or insertions (not both) along with ISI, the trellis states are defined through $D_k = (i, j)$, a length-2 vector, where i represents the number of deletions (or, insertions) up to the transmission of the i^{th} bit x_i ($i \in \{1, 2, 3, \ldots\}$), and j denotes the current channel state. For example, if n insertions ($n = 0, 1, \ldots$) occur between x_{k-1} and x_k, the state transitions from $D_{k-1} = (i, j)$ to $D_k = (i + n, j_2)$, with the corresponding output segment y_{k+i+n}^{k+i} used in branch calculations. Here, j_2 is determined probabilistically based on the channel state at time $k-1$ and the input x_k.

This trellis representation has been used to estimate achievable information rates over insertion/deletion channels with ISI using i.i.d. channel inputs. The idea is as follows. A long (simulated) codeword $\mathbf{x} = (x_1, \ldots, x_T)$, where the elements are i.i.d., is transmitted through an insertion/deletion channel with ISI, and $\mathbf{y} = (y_1, \ldots, y_R)$ is obtained. Then using the forward recursion of the BCJR algorithm, the quantity $p(\mathbf{x}, \mathbf{y})$ is estimated, which is then employed to generate an estimate of the mutual information over the channel. Using long sequences, and if needed, by averaging over multiple realizations, it is possible to obtain the achievable rates with any desired accuracy for smaller insertion/deletion probabilities. It is also possible to extend this method to Markov inputs and obtain achievable rates.

The trellis representation, with slight modifications, can also be used for decoding the inner marker codes and producing soft information about the bits (encoded by the outer code) in a concatenated coding setting over IDS channels with ISI. This is similar to the baseline methods adopted over channels with IDS only. However, the trellis complexity becomes too high due to the presence of ISI along with the insertions and deletions as implicitly implied by the trellis representation given above. Therefore, this method is not feasible for practical implementation as a decoding algorithm. As a remedy to the complexity of the standard BCJR type detection over IDS channels with ISI, deep-learning based channel detectors can be developed, which are the subject of the next section.

C. System Model and Bi-GRU Estimators for IDS Channels With ISI

The encoding follows the system models described in Section IV-A, except that bit 0 is mapped to + 1 and bit 1 is mapped to -1 prior to transmission over the IDS channel. The transmitted sequence of length T is denoted by $\mathbf{x} = (x_1, \ldots, x_T)$, where $x_k \in \{-1, +1\}$, and the received sequence by $\mathbf{y} = (y_1, \ldots, y_R)$, where $y_k \in \mathbb{R}$. The same BI-GRU estimator explained in Section IV-B is employed, retaining the same architectural details; the BI-GRU based estimator computes the posterior probabilities $P(\mathbf{s}_t \mid \mathbf{y})$ for each $\mathbf{s}_t \in \{0, \ldots, 2^S - 1\}$ and $t \in \{1, \ldots, n/S\}$ at the symbol level, forming $\mathbf{L} = (L_1, \ldots, L_{n/S})$. The input to BI-GRU at the time step t is $x_t = (y_{t-\gamma}, \ldots, y_t, \ldots, y_{t+\gamma}) \in \mathbb{R}^{2\gamma+1}$.

Similar to the networks in Section IV, the channel conditions used for training are critical and must be chosen carefully [23]. In particular, the additional parameter σ^{train} determines the SNR of the AWGN during training and affects the effectiveness of the training batches $(\mathbf{y}^{(i)}, \mathbf{m}^{(i)})$ for $i \in \{1, \ldots, B\}$, on top of the other channel conditions $\mathbf{P}_d^{\text{train}}$, $\mathbf{P}_s^{\text{train}}$, and $\mathbf{P}_i^{\text{train}}$. The importance of selecting an appropriate noise variance for the AWGN channel in order to obtain a robust deep-learning based channel decoder was investigated in [23]. The set of test SNR values is defined as $\mathbf{SNR}^{\text{test}} = \{\text{SNR}^{(1)}, \ldots, \text{SNR}^{M_{\text{SNR}}}\}$. For each training sample in a batch, a random tuple $(P_d, P_s, P_i, \text{SNR})$ is drawn from the sets $\mathbf{P}_d^{\text{train}}$, $\mathbf{P}_s^{\text{train}}$, $\mathbf{P}_i^{\text{train}}$, and $\mathbf{SNR}^{\text{train}}$, thereby completing the construction of the training pairs. Other training details are given as in Section IV-C. We employ categorical CE as the loss function.

VI. Numerical Results

In this section, we present numerical examples on the performance of the proposed symbol-level estimators. The complete experimental configurations, including channel parameters, inner and outer code settings, and BI-GRU model parameters, are summarized in Table I. Training configurations (batch size 16, learning rate 9×10^{-4} with staircase decay factor 0.9, and 30,000 steps) are fixed across all experiments and therefore not repeated in the table. For a fair comparison, the model parameter sizes of all the BI-GRU symbol-level decoders for all S are equal in terms of N, d_{MLP}, and d_{GRU}. We denote the BI-GRU decoder operating at symbol size S as *BI-GRU-S*.

We adopted three different irregular LDPC codes for the outer code with the following parameters: $(204, 102)$, $(504, 252)$, and $(816, 408)$, obtained from [46]. We provide the corresponding parity-check matrices ($\mathbf{H}$) for these LDPC codes in [47]. Testing at each point is terminated once a maximum of $30,000$ codewords or 200 frame errors is reached. The number of local iterations between the demapper and the outer-code decoder is set to $N_o = 10$, after which the outer-code decoder provides updated information back to the demapper. This exchange is repeated $M = 10$ times in total. Similarly, when the bit-level MAP decoder is used, we set the maximum number of iterations to 100 for consistency. We assume that the baseline methods fully know the channel parameters and

TABLE I

SUMMARY OF EXPERIMENTAL CONFIGURATIONS. EACH ROW CORRESPONDS TO THE SETUP USED IN THE ASSOCIATED FIGURE, AS INDICATED IN THE FIRST COLUMN. THE "FIGURE" COLUMN EXPLICITLY MAPS EACH CONFIGURATION TO THE FIGURE WHERE ITS RESULTS ARE PRESENTED. THE TABLE LISTS CHANNEL PARAMETERS, INNER AND OUTER CODE SETTINGS, AND BI-GRU MODEL PARAMETERS. TRAINING CONFIGURATIONS (BATCH SIZE 16, LEARNING RATE 9×10^{-4} WITH STAIRCASE DECAY FACTOR 0.9, AND 30,000 STEPS) ARE FIXED ACROSS ALL EXPERIMENTS AND THEREFORE NOT REPEATED HERE

Figure	**Channel Parameters**			**Inner Code**		**Outer Code**	**BI-GRU Model Parameters**				
	P_d	P_i	P_s	Marker	N_c	(n,k)	S	d_{GRU}	$\mathbf{d}_{\text{MLP}}$	γ	N_{GRU}
9	$(0.01, 0.025)$	0	0.01	$[0,1]$	30	$(504, 252)$	$\{3,5,6\}$	256	(128)	40	2
10	$(0.01, 0.03)$	0	0.01	$[0,1]$	12	$(204, 102)$	$\{2,3,4\}$	256	(128)	40	2
11	0.005	0.005	$(0.005, 0.025)$	$[0,1]$	12	$(204, 102)$	$\{2,3,4,6\}$	256	(128)	40	2
12	0.1	$(0.01, 0.03)$	0.005	$[0,1]$	12	$(204, 102)$	$\{3,4\}$	256	(128)	40	2
13	$(0.02, 0.04)$	0	0	$[0,1]$	12	$(204, 102)$	$\{2,3,4\}$	256	(128)	40	2
14	$(0.005, 0.025)$	0	0.01	$[0,1]$	30	$(816, 408)$	$\{5,6\}$	256	(128)	40	2

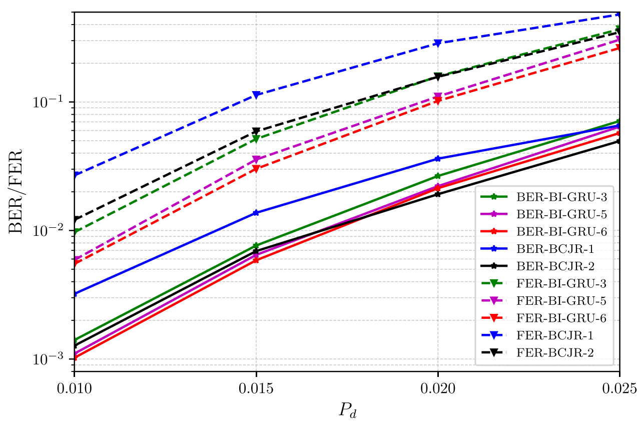


Fig. 9. BER/FER as functions of deletion probability, when channel conditions are $P_s = 0.01$ and $P_i = 0$, for $S \in \{3,5,6\}$, $N_c = 30$, marker sequence $[0,1]$, and outer LDPC code with $(504, 252)$.

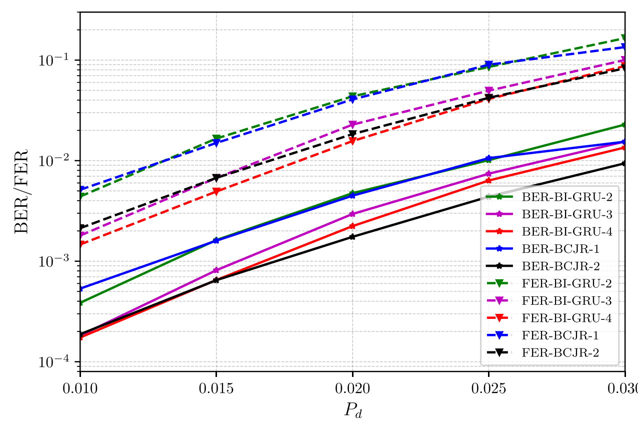


Fig. 10. BER/FER as functions of deletion probability, when channel conditions are $P_s = 0.01$ and $P_i = 0$, for $S \in \{2,3,4\}$, $N_c = 12$, marker sequence $[0,1]$ and, outer LDPC code with $(204, 102)$.

we denote baseline methods as $BCJR - S$ when performing symbol-level decoding with S bits comprising each symbol.

Selection of marker bits and their frequency and code intervals are guided by the achievable rates of the outer code for a given N_c, as established by the analysis in [19]. Some complementary results obtained with a BI-GRU estimator in [48] show similar limits for certain N_c values. Accordingly, in this paper, the marker parameters are set so that the resulting marker-code rate remains within the achievable rate bounds for the outer code.

A. Numerical Results for IDS-Only Channels

Fig. 9 compares the performance of symbol-level BI-GRU decoders for $S \in \{3,5,6\}$ with the baseline method for $S \in \{1,2\}$ for deletion probabilities $\{0.01, 0.015, 0.02, 0.025\}$ when $P_s = 0.01$ and $P_i = 0$ (i.e., no insertions), and with marker period $N_c = 30$. The outer code is the LDPC code with $n = 504$, $k = 252$. The figure shows that the FER of trained networks for symbols comprising $5, 6$ bits are clearly lower than those of the baseline algorithms with $S = 2$. Noting that, with the baseline algorithms, the complexity increases tremendously for $S > 2$ and it becomes infeasible for $S > 3$, the performance improvements offered by the BI-GRU based channel detectors are highly significant. The bit error rate (BER) of trained networks is lower than those of the baseline algorithm for deletion probabilities $P_d \in \{0.01, 0.015\}$ and slightly higher for $P_d \in \{0.02, 0.025\}$. As expected, the proposed models demonstrate consistency, as an increase in symbol length results in decreased FER and BER levels. This aligns with the assertion that performance improves for baseline and BI-GRU detectors as symbol length increases.

We also conduct experiments using a shorter LDPC code with $n = 204$, $k = 102$. In this case, the BI-GRU decoders are trained for $S \in \{2,3,4\}$. Fig. 10 compares the performance of the symbol-level BI-GRU decoders with those of the baseline methods. For this code length, too, similar trends are observed: as the symbol length S increases, the performance curves of symbol-level BI-GRU decoders improve in terms of both BER and FER. Specifically, the BI-GRU decoder with $S = 4$ for FER outperforms the baseline performance with $S = 2$. In terms of BER, an error rate of 2×10^{-4} is achieved when $P_d = 0.01$ for both BI-GRU-4 and BCJR-2. Additionally, we note that in this experiment, the P_d range is wider than in previous experiments, which impacts the training process.

Until now, we have considered the behaviors of the decoder for different deletion probabilities. We now conduct experiments to study the BI-GRU models' performance with varying P_i and P_s. Figs. 11 and 12 illustrate how symbol-level BI-GRU models perform for different values of P_s and P_i, respectively. The performance curves in Fig. 11, those for varying P_s, show that BI-GRU-3 and BI-GRU-4 outperform the bit-level baseline performance by a considerable margin

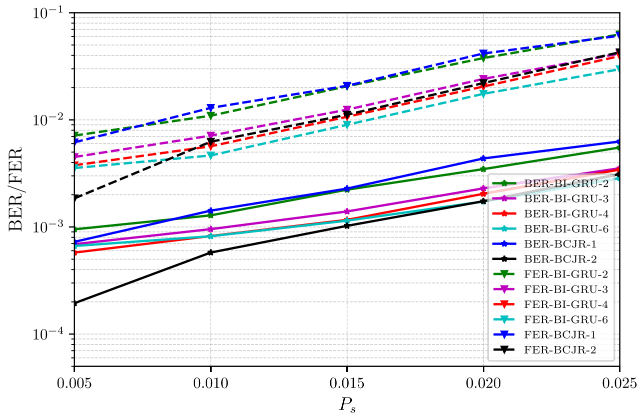


Fig. 11. BER/FER as functions of substitution probability, when channel conditions are $P_d = 0.005$ and $P_i = 0.005$, for $S \in \{2, 3, 4, 6\}$, $N_c = 12$, marker sequence $[0, 1]$ and, outer LDPC code with $(204, 102)$.

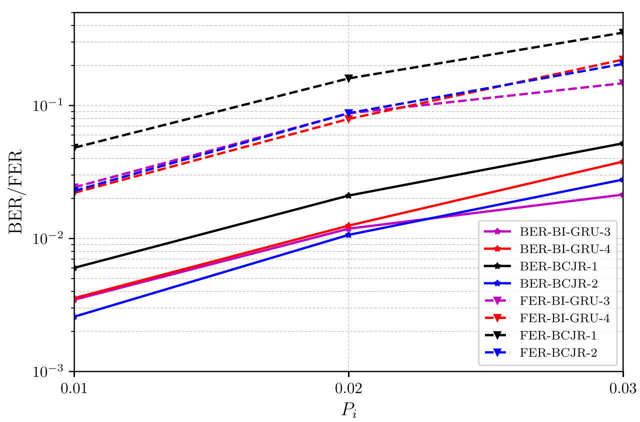


Fig. 12. BER/FER as functions of insertion probability, when channel conditions are $P_d = 0.01$ and $P_s = 0.005$, for $S \in \{3, 4\}$, $N_c = 12$, marker sequence $[0, 1]$, and, outer LDPC code with $(204, 102)$.

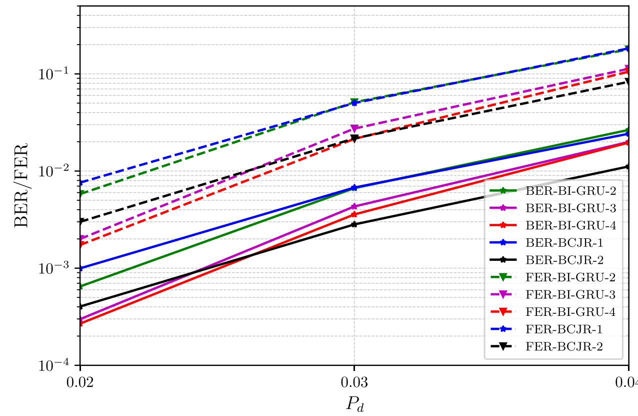


Fig. 13. BER/FER as functions of deletion probability, when channel conditions are $P_s = 0.01$ and $P_i = 0$, for $S \in \{2, 3, 4\}$, $N_c = 12$, marker sequence $[0, 1]$, and outer LDPC code with $(204, 102)$.

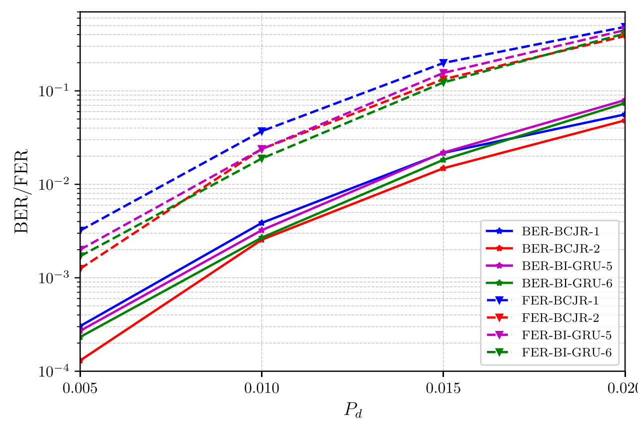


Fig. 14. BER/FER as functions of deletion probability, when channel conditions are $P_s = 0.01$ and $P_i = 0$, for $S \in \{5, 6\}$, $N_c = 30$, marker sequence $[0, 1]$ and outer LDPC code with $(816, 408)$.

in terms of both the BER and FER. Additionally, BI-GRU-4 and BI-GRU-6 surpass the 2-bit-level baseline performance, except at a substitution probability of 5×10^{-3} for FER. This experiment highlights that symbol-level BI-GRU decoders can be trained and perform well not only for various deletion probabilities but also for varying substitution probabilities.

Similarly, the performance curves in Fig. 12, those for varying P_i, demonstrate that BI-GRU-3 and BI-GRU-4 outperform the bit-level baseline performance in terms of both the BER and FER. BI-GRU-4 also outperforms the 2-bit-level baseline performance for insertion probabilities of 0.01 and 0.02, but it remains inferior at $P_i = 0.03$. The final experiments are only for deletion channels without substitutions or insertions. Fig. 13 shows the error curves for deletion probabilities of 0.02, 0.03, and 0.04. Fig. 14 is an experiment with a longer LDPC code, with $n = 816$, $k = 408$. Moreover, the number of steps required by the BI-GRU estimator decreases as S increases. For example, if we decode a code with $T = 600$, the bit-level BI-GRU decoder requires 600 steps, BI-GRU-2 requires 300 steps, and BI-GRU-6 requires 100 steps. Therefore, for bit-level decoders in [1], the length of the LDPC code is limited by the sequence length. In contrast, by increasing S, longer LDPC codes can be decoded efficiently, enabling the use of higher code lengths due to the smaller time steps.

The above results demonstrate that the symbol-level decoders provide solutions for a number of bits in each symbol up to 6, exceeding the threshold for the baseline symbol-level decoders (which were limited to $S \leq 3$). Furthermore, we observe that BI-GRU decoders exhibit strong performance across different ranges of the channel parameters. For further results, the reader is referred to Chapter 4 of the first author's M.S. thesis [48].

Increasing S in deep learning decoders beyond some threshold presents significant challenges as well, as in the case of symbol-level MAP detectors. As S increases, the number of classes in the final layer of the symbol-level BI-GRU detector increases exponentially to 2^S. While theoretically, performance improves with a higher S, practical constraints in deep learning models limit this growth.

Although the experimental results show that the proposed method achieves noticeable performance gains only under a subset of the evaluated settings, these findings underscore its value as a scalable alternative when exact MAP decoding is difficult or intractable. In particular, by operating on longer symbols than conventional MAP decoders [19] (which can be up to 3-bit symbols), the model can leverage dependencies that are otherwise inaccessible, which is especially useful for

TABLE II

SUMMARY OF EXPERIMENTAL CONFIGURATIONS FOR IDS CHANNELS WITH ISI. EACH ROW CORRESPONDS TO ONE EXPERIMENTAL SETUP, INCLUDING CHANNEL PARAMETERS, INNER AND OUTER CODE SETTINGS, AND BI-GRU MODEL PARAMETERS. THE FIRST COLUMN ("FIGURE") EXPLICITLY INDICATES THE FIGURE IN WHICH THE RESULTS OF THE CORRESPONDING EXPERIMENT ARE PRESENTED, ALLOWING READERS TO EASILY TRACE EACH CONFIGURATION TO ITS VISUAL REPRESENTATION. THE ISI CHANNEL IS FIXED TO PR4 WITH IMPULSE RESPONSE $\mathbf{h} = [1/\sqrt{2}, 0, -1/\sqrt{2}]$. TRAINING CONFIGURATIONS (BATCH SIZE 16, LEARNING RATE 9×10^{-4} WITH A STAIRCASE DECAY FACTOR OF 0.9, 30,000 STEPS, AND GRADIENT CLIPPING WITH NORM 1) ARE FIXED ACROSS ALL EXPERIMENTS AND THEREFORE NOT REPEATED HERE

Figure	Channel Parameters			Inner Code		Outer Code	BI-GRU Model Parameters				
	P_d	P_i	P_s	Marker	N_c	(n, k)	S	d_{GRU}	$\mathbf{d}_{\text{MLP}}$	γ	N
17	0.01	0	$\{0, 0.01\}$	$[0, 1, 0]$	6	$(204, 102)$	2	512	128	40	4
18	$\{0.01, 0.02\}$	0	0.01	$[0, 1, 0]$	6	$(204, 102)$	2	512	128	40	4
19	0.01	0	0	$[0, 1, 0]$	$\{6, 12\}$	$(204, 102)$	2	512	128	40	4

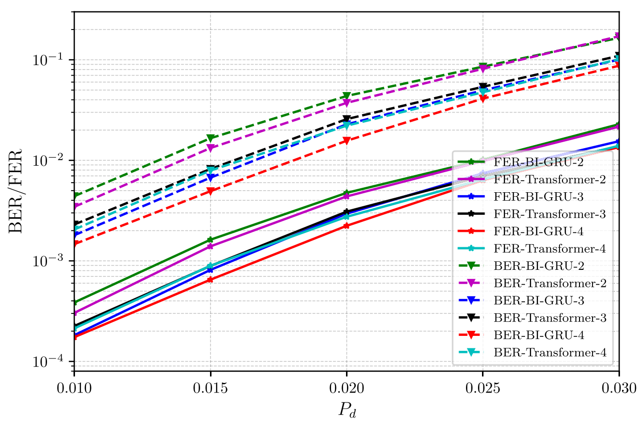


Fig. 15. Comparison of transformer-based symbol-level decoders and BI-GRU based symbol-level decoders. BER/FER as functions of deletion probability, when channel conditions are $P_s = 0.01$ and $P_i = 0$, for $S \in \{2, 3, 4\}$, $N_c = 12$, marker sequence $[0, 1]$ and, outer LDPC code with $(204, 102)$.

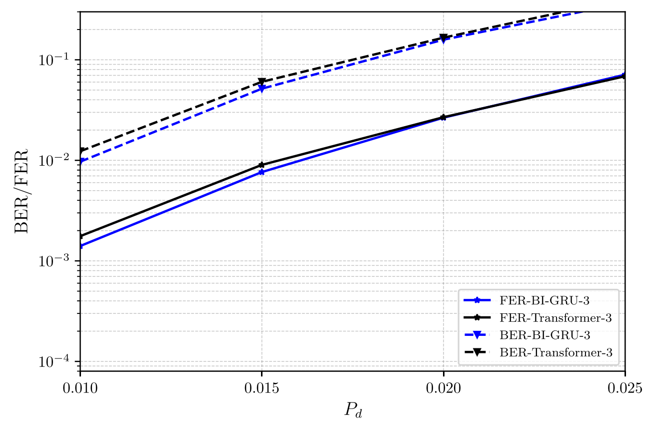


Fig. 16. Comparison of transformer-based symbol-level decoders and BI-GRU based symbol-level decoders for longer LDPC code. BER/FER as functions of deletion probability, when channel conditions are $P_s = 0.01$ and $P_i = 0$, for $S \in \{3, 5, 6\}$, $N_c = 30$, marker sequence $[0, 1]$, and outer LDPC code with $(504, 252)$.

channels with synchronization errors and additional ISI/IDS impairments. This scalability also makes the approach practical for applications such as DNA storage, where analytical models and tractable MAP rules are limited.

As a final note, the BI-GRU is not the only possible deep learning model for addressing the synchronization errors. For instance, architectures such as transformers can also be used as alternative solutions, as also indicated in the recently published work on trace reconstruction [49]. The authors in [49] demonstrate that transformer architectures are suitable for trace reconstruction tasks in DNA storage, where multiple noisy copies of a strand are received through insertions/deletions. Therefore, we also conducted experiments with transformer-based symbol-level decoders under the same parameters as the BI-GRU networks for a fair comparison. In the transformer architecture, the attention head size is set to 12 and the dropout rate is selected as 0.1. Specifically, we performed two experiments using two different LDPC codes: one with code parameters $(204, 102)$ and another with $(504, 252)$. For the first experiment, whose results are depicted in Fig. 15, we used the channel parameters from Fig. 10 ($P_s = 0.01$, $P_i = 0$, and $P_d \in [0.01, 0.03]$ with increments of 0.005) and evaluated the cases with symbol lengths $S \in \{2, 3, 4\}$. The results show that the transformer shows competitive performance for small symbol lengths, particularly for $S = 2$, however, its performance degrades as S increases. For the second example, Fig. 16 depicts the results with the longer LDPC code $(504, 252)$ with only $S = 3$ (as training with $S = 5$ in this case did not give a suitable result for the transformer architecture). The results show that the transformer architecture offers similar performance for $S = 3$, while not being suitable for $S = 5$. In contrast, the BI-GRU architecture remains robust and trainable even when the symbol length is increased to $S = 5$. In summary, the BI-GRU and transformer models exhibit similar performance for a small number of jointly considered symbols, while the transformer performance worsens as this number increases.

B. Numerical Results for IDS Channels With ISI

In this section, we provide numerical results for BI-GRU based receivers for concatenated codes over insertion/deletion channels with ISI. A commonly used channel model in recording systems, the partial response-4 (PR4) target, is employed throughout all the experiments in this section. The channel response is given by $\mathbf{h} = \left[\frac{1}{\sqrt{2}},\ 0,\ -\frac{1}{\sqrt{2}}\right]$ [45]. The model sizes, together with the complete experimental configurations for this section, are summarized in Table II, which also highlights their increase compared to the BI-GRU models designed for IDS channels without ISI (Table I). This adjustment is necessary because of the challenging nature of the channel,

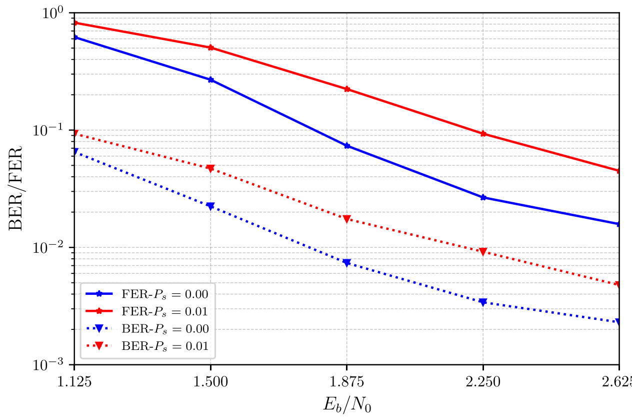


Fig. 17. BER/FER as functions of E_b/N_0, when channel conditions $P_d = 0.01$ and $P_s \in \{0, 0.01\}$, for $S = 2$, $N_c = 6$, marker sequence $[0, 1, 0]$, and, outer LDPC code with $(204, 102)$.

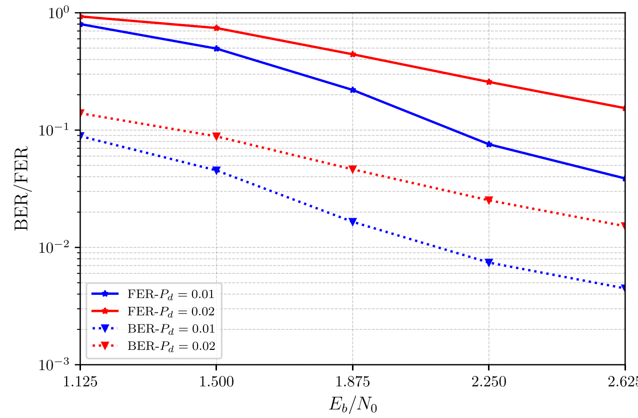


Fig. 18. BER/FER as functions of E_b/N_0, when channel conditions $P_s = 0.01$ and $P_d \in \{0.01, 0.02\}$, for $S = 2$, $N_c = 6$, marker sequence $[0, 1, 0]$ and, outer LDPC code with $(204, 102)$.

which is influenced by ISI channel taps and the presence of Gaussian noise. Gradient clipping with a norm of 1 is applied to control gradient magnitudes. Training the BI-GRU models for these channels is significantly more difficult than for the models in the previous section, requiring a parameter search for the learning rate and batch size. The parameters used for training include $N_c \in \{6, 12\}$, $\gamma = 3$, and a marker sequence of $[0, 1, 0]$. The networks are trained for 30,000 steps.

Experiments are conducted to select the best value of γ among different values of the parameter $\gamma \in \{5, 10, 20, 40\}$ [48]. We found that, for these examples, the model with $\gamma = 40$ outperforms the others in terms of BER and FER, and thus this value is adopted in the subsequent settings.

Fig. 17 presents the performance of the proposed decoder under different channel conditions. The first scenario is the deletion channel with $P_d = 0.01$, while the second one involves a channel with both substitutions and deletions with $P_d = 0.01$ and $P_s = 0.01$. The horizontal axis represents the E_b/N_0, which differs from the earlier figures where the axis typically represented the range of deletion or insertion probabilities, or other channel parameters. We employ symbol-level deep learning architectures with two bits per symbol ($S = 2$). As expected, the model performs better under more favorable channel conditions, with the deletion-only channel with $P_d = 0.01$ outperforming the channel having $P_d = 0.01$ and $P_s = 0.01$ for all the E_b/N_0 values considered. A BER of 10^{-2} is achieved at an E_b/N_0 of 2.25 dB for the BI-GRU decoder on the channel with $P_d = 0.01$ and $P_s = 0.01$, and at an E_b/N_0 of 1.7 dB for the BI-GRU decoder over the deletion channel with $P_d = 0.01$.

Fig. 18 shows the performance of the proposed decoder for two different deletion probabilities $P_d = 0.01$, and $P_d = 0.02$, while fixing the substitution probability at $P_s = 0.01$. As expected, the performance of the model for more favorable channel conditions is superior for all the E_b/N_0 values considered. A BER of 5×10^{-2} is achieved at an E_b/N_0 of 2.05 dB for BI-GRU based receiver for the channel with $P_d = 0.01$ and $P_s = 0.01$; while an E_b/N_0 of 2.25 dB is needed for the channel with $P_d = 0.02$ and $P_s = 0.02$.

Lastly, Fig. 19 shows the performance of the proposed decoder for $N_c = 6$ and $N_c = 12$ under the same channel conditions for both models, i.e., for a deletion-only channel with $P_d = 0.01$.

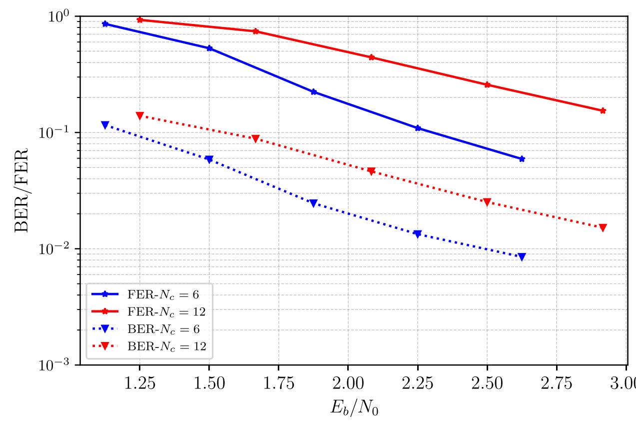


Fig. 19. BER/FER as functions of E_b/N_0, when channel condition $P_d = 0.01$, for $S = 2$, $N_c \in \{6, 12\}$, marker sequence $[0, 1, 0]$ and, outer LDPC code with $(204, 102)$.

The numerical results demonstrate the powerful decoding capabilities of BI-GRU based detectors for concatenated codes over IDS channels with ISI, a scenario that has proven challenging in terms of required complexity using traditional approaches. Although the existing methods discussed in Section V-B can be adapted to the concatenated code setting where inner codes serve as markers, implementing MAP detection for these channels becomes highly complex, even at the bit level, it is simply impractical. Hence, no direct performance comparisons are provided. We conclude this section by noting that our results may pave the way for applying deep-learning based estimators to other variations of IDS channels as well.

VII. Conclusion

This paper presents symbol-level deep-learning based decoders for concatenated codes, where the inner code is a marker code, applied over IDS channels. Symbol-level decoders exploit the correlations among neighboring bits that are affected by IDS channels, resulting in improved performance. However, limitations arise in the baseline symbol-level MAP decoder, which can group up to three bits due to increased complexity. In contrast, deep-learning based symbol-level decoders can handle longer symbols exceeding this threshold, that is, they can be used even when the common baseline technique is not feasible. Symbol-level deep learning decoders represent improvements over their bit-level counterparts and improve the error rate performance while keeping the complexity at a reasonable level. We have also obtained results for IDS channels which are further degraded by ISI. Our results demonstrate that deep-learning based methods can be used to produce soft information even for these challenging models, whereas the traditional algorithms are too complex due to the tremendous increase in the number of states in the required joint trellis representation. Future research directions include pruning and quantization techniques to decrease network size and inference times, extending the proposed architectures to applications in DNA data storage, and developing reinforcement learning–based solutions for channel detection and decoding.

ACKNOWLEDGMENT

This work was funded by the European Union through the ERC Advanced Grant 101054904: TRANCIDS. Views and opinions expressed are, however, those of the authors only and do not necessarily reflect those of European Union or European Research Council Executive Agency. Neither European Union nor the granting authority can be held responsible for them.

The work of E. Uras Kargı was completed when he was with the Department of Electrical and Electronics Engineering, Bilkent University, Ankara, Türkiye.

REFERENCES

[1] E. U. Kargı and T. M. Duman, "A deep learning based decoder for concatenated coding over deletion channels," in *Proc. IEEE Int. Conf. Commun.*, Jun. 2024, pp. 2797–2802.

[2] S. M. H. T. Yazdi, H. M. Kiah, E. Garcia-Ruiz, J. Ma, H. Zhao, and O. Milenkovic, "DNA-based storage: Trends and methods," *IEEE Trans. Mol., Biol. Multi-Scale Commun.*, vol. 1, no. 3, pp. 230–248, Sep. 2015.

[3] R. L. White, R. M. H. Newt, and R. F. W. Pease, "Patterned media: A viable route to 50 Gbit/in^2 and up for magnetic recording?," *IEEE Trans. Magn.*, vol. 33, no. 1, pp. 990–995, Jan. 1997.

[4] H. Richter et al., "Recording potential of bit-patterned media," *Appl. Phys. Lett.*, vol. 88, no. 22, pp. 1–3, May 2006.

[5] L. Conde-Canencia and L. Dolecek, "Nanopore DNA sequencing channel modeling," in *Proc. IEEE Int. Workshop Signal Process. Syst. (SiPS)*, Mar. 2019, pp. 258–262.

[6] J. Hu, T. M. Duman, M. F. Erden, and A. Kavcic, "Achievable information rates for channels with insertions, deletions, and intersymbol interference with I.I.D. inputs," *IEEE Trans. Commun.*, vol. 58, no. 4, pp. 1102–1111, Apr. 2010.

[7] R. L. Dobrushin, "Shannon's theorems for channels with synchronization errors," *Problems Inf. Transmiss.*, vol. 3, no. 4, pp. 11–26, 1967.

[8] D. Fertonani and T. M. Duman, "Novel bounds on the capacity of the binary deletion channel," *IEEE Trans. Inf. Theory*, vol. 56, no. 6, pp. 2753–2765, Jun. 2010.

[9] D. Fertonani, T. M. Duman, and M. F. Erden, "Bounds on the capacity of channels with insertions, deletions and substitutions," *IEEE Trans. Commun.*, vol. 59, no. 1, pp. 2–6, Jan. 2011.

[10] M. Rahmati and T. M. Duman, "Bounds on the capacity of random insertion and deletion-additive noise channels," *IEEE Trans. Inf. Theory*, vol. 59, no. 9, pp. 5534–5546, Sep. 2013.

[11] D. Fertonani and T. M. Duman, "Upper bounding the deletion channel capacity by auxiliary memoryless channels," in *Proc. IEEE Int. Conf. Commun.*, Jun. 2009, pp. 1–5.

[12] M. Rahmati and T. M. Duman, "Upper bounds on the capacity of deletion channels using channel fragmentation," *IEEE Trans. Inf. Theory*, vol. 61, no. 1, pp. 146–156, Jan. 2015.

[13] R. R. Varshamov and G. M. Tenengolts, "Codes which correct single asymmetric errors," *Autom. Remote Control*, vol. 26, no. 2, pp. 286–290, 1965.

[14] W. Ferreira, W. Clarke, A. Helberg, K. Abdel-Ghaffar, and A. Vinck, "Insertion/deletion correction with spectral nulls," *IEEE Trans. Inf. Theory*, vol. 43, no. 2, pp. 722–732, Feb. 1997.

[15] M. F. Mansour and A. H. Tewfik, "Convolutional codes for channels with substitutions, insertions, and deletions," in *Proc. IEEE Global Telecommun. Conf., IEEE GLOBECOM 02*, vol. 2, Nov. 2002, pp. 1051–1055.

[16] T. G. Swart and H. C. Ferreira, "Insertion/deletion correcting coding schemes based on convolution coding," *Electron. Lett.*, vol. 38, no. 16, pp. 871–873, Aug. 2002.

[17] M. C. Davey and D. J. C. Mackay, "Reliable communication over channels with insertions, deletions, and substitutions," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 687–698, Feb. 2001.

[18] E. A. Ratzer and D. J. MacKay, "Codes for channels with insertions, deletions and substitutions," in *Proc. 2nd Int. Symp. Turbo Codes Rel. Topics*, 2000, pp. 149–156.

[19] F. Wang, D. Fertonani, and T. M. Duman, "Symbol-level synchronization and LDPC code design for insertion/deletion channels," *IEEE Trans. Commun.*, vol. 59, no. 5, pp. 1287–1297, May 2011.

[20] R. Shibata, G. Hosoya, and H. Yashima, "Design and construction of irregular LDPC codes for channels with synchronization errors: New aspect of degree profiles," *IEICE Trans. Fundamentals Electron., Commun. Comput. Sci.*, vol. E103.A, no. 10, pp. 1237–1247, 2020.

[21] R. Shibata and H. Yashima, "Windowed-based synchronization error-correction for spatially coupled codes," *IEICE Commun. Exp.*, vol. 11, no. 11, pp. 697–702, 2022.

[22] R. Shibata, G. Hosoya, and H. Yashima, "Joint iterative decoding of spatially coupled LDPC codes for position errors in racetrack memories," *IEICE Trans. Fundamentals Electron., Commun. Comput. Sci.*, vols. E101–A, no. 12, pp. 2055–2065, 2018.

[23] T. Gruber, S. Cammerer, J. Hoydis, and S. T. Brink, "On deep learning-based channel decoding," in *Proc. 51st Annu. Conf. Inf. Sci. Syst. (CISS)*, 2017, pp. 1–6.

[24] H. Kim, Y. Jiang, R. Rana, S. Kannan, S. Oh, and P. Viswanath, "Communication algorithms via deep learning," 2018, *arXiv:1805.09317*.

[25] Y. Jiang, S. Kannan, H. Kim, S. Oh, H. Asnani, and P. Viswanath, "DEEPTURBO: Deep turbo decoder," in *Proc. IEEE 20th Int. Workshop Signal Process. Adv. Wireless Commun. (SPAWC)*, Jul. 2019, pp. 1–5.

[26] Y. Choukroun and L. Wolf, "Error correction code transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 38695–38705.

[27] M. Levy, Y. Choukroun, and L. Wolf, "Accelerating error correction code transformers," 2024, *arXiv:2410.05911*.

[28] Y. Liao, S. A. Hashemi, J. M. Cioffi, and A. Goldsmith, "Construction of polar codes with reinforcement learning," *IEEE Trans. Commun.*, vol. 70, no. 1, pp. 185–198, Jan. 2022.

[29] S. K. Mishra, D. Katyal, and S. A. Ganapathi, "A modified Q-learning algorithm for rate-profiling of polarization adjusted convolutional (PAC) codes," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Apr. 2022, pp. 2363–2368.

[30] H.-Y. Kwak, D.-Y. Yun, Y. Kim, S. Kim, and J. No, "Boosting learning for LDPC codes to improve the error-floor performance," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 22115–22131.

[31] S.-J. Park, H.-Y. Kwak, S.-H. Kim, Y. Kim, and J.-S. No, "CrossMPT: Cross-attention message-passing transformer for error correcting codes," 2024, *arXiv:2405.01033*.

[32] Y. Choukroun and L. Wolf, "A foundation model for error correction codes," in *Proc. Int. Conf. Represent. Learn.*, vol. 2024, 2024, pp. 2153–2171.

[33] Y. Choukroun and L. Wolf, "Learning linear block error correction codes," in *Proc. 41st Int. Conf. Mach. Learn.*, vol. 235. PMLR, Jul. 2024, pp. 8801–8814.

[34] A. J. X. Guo, M. Wei, Y. Dai, Y. Wei, and P. Zhang, "Disturbance-based discretization, differentiable IDS channel, and an IDS-correcting code for DNA-based storage," 2024, *arXiv:2407.18929*.

[35] R. G. Gallager, "Sequential decoding for binary channels with noise and synchronization errors," Massachusetts Inst. Technol., Lincoln Lab., Tech. Rep. 25G 2, Oct. 1961.

[36] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997.

[37] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," 2014, *arXiv:1406.1078*.

[38] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," 2012, *arXiv:1211.5063*.

[39] S. Cammerer, J. Hoydis, F. A. Aoudia, and A. Keller, "Graph neural networks for channel decoding," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Dec. 2022, pp. 486–491.

[40] G. Ma, X. Jiao, J. Mu, H. Han, and Y. Yang, "Deep learning-based detection for marker codes over insertion and deletion channels," *IEEE Trans. Commun.*, vol. 72, no. 10, pp. 5945–5959, Oct. 2024.

[41] F. Wang, "Coding for Insertion/Deletion Channels," Ph.D. thesis, School Elect., Comput. Energy Eng., Arizona State Univ., Tempe, AZ, USA, 2012.

[42] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.

[43] K. You, M. Long, J. Wang, and M. I. Jordan, "How does learning rate decay help modern neural networks?," 2019, *arXiv:1908.01878*.

[44] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Int. Conf. Mach. Learn. -, vol. 37*, 2015, p. 448.

[45] J. Hu, T. M. Duman, E. M. Kurtas, and M. F. Erden, "Bit-patterned media with written-in errors: Modeling, detection, and theoretical limits," *IEEE Trans. Magn.*, vol. 43, no. 8, pp. 3517–3524, Aug. 2007.

[46] D. J. MacKay. (2025). *Encyclopedia of Sparse Graph Codes: Codes/Data*. [Online]. Available: https://www.inference.org.uk/mackay/codes/data.html

[47] E. U. Kargı. (2025). *Symbol-Level BIGRU Decoders for IDS Channels*. [Online]. Available: https://github.com/Bilkent-CTAR-Lab/Symbol-Level-BIGRU-Decoders-for-IDS-Channels

[48] E. U. Kargı, "Deep learning based decoders for concatenated codes over insertion and deletion channels," M.S. thesis, Dept. Elect. Electron. Eng., Bilkent Univ., Ankara, Türkiye, 2025.

[49] J. Streit, F. Weindel, and R. Heckel, "Transformer-based decoding in concatenated coding schemes under synchronization errors," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2025, pp. 1–6.

E. Uras Kargı received the B.Sc. and M.Sc. degrees in electrical and electronics engineering from Bilkent University, Ankara, Türkiye, in 2022 and 2025, respectively. He is currently pursuing the Ph.D. degree with the Networked Systems Group, Faculty of Electrical Engineering, Mathematics and Computer Science (EEMCS), TU Delft, The Netherlands.

His research interests broadly include machine learning applications for digital communications and networked systems, online learning, deep learning, reinforcement learning, and channel coding.

Tolga M. Duman (Fellow, IEEE) received the B.S. degree in electrical engineering from Bilkent University, Ankara, Türkiye, in 1993, and the M.S. and Ph.D. degrees in electrical engineering from Northeastern University, Boston, MA, USA, in 1995 and 1998, respectively.

Before joining Bilkent University in September 2012, he was a Full Professor with the School of ECEE, Arizona State University. He is currently a Professor with the Electrical and Electronics Engineering Department, Bilkent University. His current research interests are in systems, with a particular focus on communications and signal processing, including wireless and mobile communications, channel coding/modulation, coding for wireless communications, information theory, and data storage systems, machine learning for communications, and semantic communications/signal processing. He was a recipient of the National Science Foundation CAREER Award, the IEEE Third Millennium medal, and European Research Council (ERC) Advanced Grant. He served on the editorial board for various journals, including IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS and IEEE COMMUNICATIONS SURVEYS AND TUTORIALS. He also served as the Editor-in-Chief for *Physical Communication* (Elsevier) and IEEE TRANSACTIONS ON COMMUNICATIONS.